XP Driver Developer's Guide

Release Runtime v25

Remote Technologies Inc.

Apr 25, 2022

This PDF version of the XP Driver Developer's Guide is provided as a convenience and may not contain the latest content. The latest version can always be found online at https://developer.rticontrol.com/

The latest version of this guide may be accessed at https://developer.rticontrol.com/.

Revision 6983207

Contents

1 Introduction **1**
- 1.1 Prerequisites . . . 1
- 1.2 Two-Way System Overview . . . 2
- 1.3 System Variables . . . 2
- 1.4 Development Process . . . 3
- 1.5 Samples . . . 4

2 Autoprogramming **5**
- 2.1 Overview . . . 5
- 2.2 Tags . . . 5
- 2.3 Tag Parameters . . . 6
- 2.4 Templates . . . 7
- 2.5 Multi-Instance Drivers . . . 7

3 Scripting Concepts **13**
- 3.1 Event-Driven Programming . . . 13
- 3.2 Debugging . . . 13
- 3.3 Extensibility . . . 14
- 3.4 Views . . . 14
- 3.5 Item Lists . . . 16
- 3.6 Memory Usage . . . 18

4 Driver Info (Help) **19**

5 Tools **20**
- 5.1 PackageDriver . . . 20
- 5.2 TraceViewer . . . 21

6 Driver XML Reference **22**
- 6.1 General XML Concepts . . . 22
- 6.2 DriverManifest.xml . . . 25
- 6.3 DeviceDescription.xml . . . 28
- 6.4 ConfigSettings.xml . . . 30

6.5 SystemVariables.xml . . . 34
6.6 SystemFunctions.xml . . . 36
6.7 SystemEvents.xml . . . 40

7 RTI Script Reference **42**
7.1 System Object . . . 43
7.2 Config Object . . . 52
7.3 Comm Object . . . 53
7.4 Serial Object . . . 57
7.5 TCP Object . . . 59
7.6 TCPServer Object . . . 63
7.7 HTTP Object . . . 67
7.8 UDP Object . . . 76
7.9 MulticastUDP Object . . . 78
7.10 Timer Object . . . 80
7.11 ScheduledEvent Object . . . 83
7.12 SystemVars Object . . . 87
7.13 SystemVarsList Object . . . 90
7.14 Persistence Object . . . 97
7.15 Util Object . . . 98
7.16 Crypto Object . . . 99
7.17 Ping Object . . . 107
7.18 SSH Object . . . 112
7.19 MDNS Object . . . 119

Index **123**

CHAPTER 1

Introduction

This guide describes the functioning and creation of device drivers for the RTI XP series control processors. The RTI two-way driver architecture was designed to fully separate the creation and maintenance of the user interface from that of the driver, so that a single driver can be used to provide information to the full range of RTI products. Drivers are only responsible for communicating with their controlled product, typically via RS-232 or TCP/IP. All communication between the XP and the RTI interface devices (via Ethernet, ZigBee, RS-485, or Internet) is handled by the RTI runtime system with no involvement from the driver.

Each driver consists of code that implement the driver's functions and metadata that describes the driver's capabilities to the Integration Designer programming environment. Integration Designer relies on detailed metadata describing each system variable, function, and event supported by the driver to present a simple programming interface to the dealers. A typical driver package contains as much metadata as it does driver code.

1.1 Prerequisites

It is assumed that the driver writer has a thorough understanding of the operation of RTI remotes, keypads, and processors, and how to program them with Integration Designer. The drivers are written in a dialect of the JavaScript programming language, so the driver writer must be familiar with JavaScript programming. The XP includes the Mozilla JavaScript engine version 1.7, including the E4X XML processing API. Mozilla maintains a web site at https://developer.mozilla.org/en-US/docs/Web/JavaScript that contains more information on the specific features available in their implementation. *JavaScript: The Definitive Guide (5th Edition)* by David Flanagan, published by O'Reilly Media, is an excellent reference for the JavaScript language.

1.2 Two-Way System Overview

Before creating drivers, it is important to have a good understanding of the parts of the RTI two-way system and how they interact with one another. The entire two-way system is based around the concept of the *system variable*, which is an individual piece of data provided by a driver. The drivers provide this information without dictating how it is presented by the interfaces (touch panels and remotes). A single system variable can be displayed as a number on one interface, a bar graph on another, and a text string on a third interface. Each interface notifies the XP of the system variables it is interested in (it *subscribes* to them) each time the page changes on the interface, and it *unsubscribes* to variables it is no longer interested in. The system variable database on the XP tracks which variables are in use on each interface, and automatically notifies each interface whenever the value of the variable changes.

Communications in the opposite direction, from interface to driver, are also supported. The driver may export a number of *system functions* that can be called by button presses or macros. There are two types of function calls, *static* and *dynamic*. Static function calls are used when all of the parameters are known at design time, and dynamic function calls are used when parameters can vary at run time (such as those attached to slider objects on the interface). This allows System Functions to be called statically even by devices without two-way capabilities (such as existing RTI 433MHz remotes and IR keypads). The static / dynamic distinction is made by Integration Designer and is completely transparent to the driver.

Finally, the two-way system provides a *system event* mechanism that allows dealers to customize the actions taken by drivers without actually editing the driver code. When the driver detects a condition that may be of interest to users of the driver, it can signal an event, which can be hooked up to a system macro with Integration Designer.

1.3 System Variables

There are three general-purpose types of system variables: Boolean, Integer, and String. There are also two special-purpose types: Item List and Dynamic Image.

1.3.1 Boolean

Boolean system variables indicate a true / false condition (such as on/off, open/closed, muted/unmuted, etc). In Integration Designer, boolean variables can be used for button text, for the reversed / disabled / visible button properties, and as the data source for toggle button graphics. They can also be tested from within macros using the System Variable Test macro step.

1.3.2 Integer

Integer system variables store a 32 bit signed value, with a range from -2,147,483,648 to 2,147,483,647. Some interfaces, notably the RKM-1, RK1, and RK2 can only handle 16 bit integers, so if compatibility with these products is desired, drivers should limit their integer values to the range -32,768 to 32,767. In Integration Designer, integer variables can be used as button text, either directly, formatted as a fixed-point number, or as an index into a string lookup table. They can also be used as the data source for graph and image list objects.

1.3.3 String

String system variables store a Unicode string value. The two-way system fully supports non-Latin alphabets, but the display of such text depends on the presence of appropriate fonts on the interface.

1.3.4 Item List

Item List system variables support the scrolling list functionality of interface devices.

1.3.5 Dynamic Image

Dynamic Image system variables provide a mechanism to allow drivers to specify an image that is displayed on compatible interfaces (such as cover art, weather maps, and other changing images). Dynamic Images provide an HTTP URL of the image to the interface, which then connects directly to the image source to retrieve and display the image. Since the interface is responsible for retrieving the image data from the URL, only devices with a wired or wireless Ethernet connection can currently make use of dynamic images. Current firmware versions for all interfaces support JPEG, GIF and animated GIF images.

1.4 Development Process

1. Create a new driver ID using the *PackageDriver -i* command.
2. Create scripts and XML files in a text editor.
3. Use **PackageDriver** to generate an RTIDRIVER file from the scripts and XML files.
4. Create an Integration Designer file containing an XP processor and at least one interface.
5. Load the driver onto the XP and create test pages on the interface.
6. Download the program to the XP and interface.
7. Test the functionality of the driver. The **TraceViewer** tool can be used to monitor debug output from the driver.

8. Make any necessary changes to the scripts and/or XML files, and then build a new driver with **`PackageDriver`**.
9. Use the Update Driver command in Integration Designer to load the new driver into the data file.
10. Repeat steps 6-9 as necessary.

The *PackageDriver* and *TraceViewer* tools are described in chapter 6 of this guide.

1.5 Samples

This guide is accompanied by a number of sample drivers. The drivers in the `sample\` folder are simple drivers intended to demonstrate individual features of the two-way driver system. The drivers in the `release\` folder contain the source code to fully-functional drivers that RTI has released. When using any of these folders as a starting point for a new driver, **it is critical to generate a new ID and replace the existing one in the DriverManifest.xml file before starting work on the new driver.** Each driver MUST have a unique ID; several mechanisms in Integration Designer rely on this.

CHAPTER 2

Autoprogramming

2.1 Overview

Integration Designer 10 adds extensive new automatic programming capabilities. As a driver developer, there are a few things you must do in your driver to ensure that hooks seamlessly into Integration Designer's autoprogramming system. Since the autoprogramming system is a design-time feature, the changes are centered around the XML files that describe your driver to Integration Designer. The JavaScript source files should need no modification.

The changes are designed to allow you to add autoprogramming capability to an existing driver without making it incompatible with previous versions of Integration Designer, so you can maintain a single driver that is usable with Integration Designer 9 and 10. Because of this, the existing `minimumSoftwareVersion` attribute in the *`driver`* element is ignored by Integration Designer 10, so that the value can continue to be set appropriately based on the Integration Designer 9-level features that the driver uses. Integration Designer 10 introduces the new `minimumApexVersion` attribute that serves the same purpose for newer versions of Integration Designer. Drivers that include autoprogramming functionality must set the `minimumApexVersion` attribute to `10.0` (or higher). If this is not set, the driver cannot use any of the autoprogramming extensions.

2.2 Tags

The autoprogramming system in Integration Designer 10 is based on the concept of a *tag*. Rather than directly assigning commands and macros to buttons, as was done in Integration Designer 9, each button on every device has a tag. The tag is a simple text string, which is matched with tags declared by drivers to determine which command is assigned to each button. This resolution can happen at multiple levels (per-source, per-device, and per-room) in Integration Designer, making it easy to reprogram entire systems with only a few clicks.

Note: In order for this system to work reliably, it is important that all drivers use the same tags to describe the same functionality, so that templates that have been properly tagged will connect with the functions exported by those drivers.

A list of standard tags can be found at the RTI Developer Site. All drivers should use these tags consistently to describe their functionality. If you have functions which are not covered by the standard tag lists, please suggest new additions there.

It is not an error to use tags not on the standard list, but *PackageDriver* will warn if you do so. Using non-standard tags means that none of the existing templates will autoprogram those functions, so please try to use the existing tags for the functions described if at all possible.

Every function and variable that the driver provides can have a `buttontag` attribute, which specifies the tag in the template with which the function or variable will match.

2.3 Tag Parameters

New in Integration Designer 11.0

Integration Designer 11 adds a new feature for tags called *tag parameters*. Tags using this feature should be of the form *Tagname*:#PARAMETER[1-4], where *Tagname* is any tag name that you create, followed by exactly one of **:#PARAMETER1** through **:#PARAMETER4**. This type of tag is only valid on functions in *SystemFunctions.xml*, and must be attached to a function that takes at least one string parameter. At download time, the string after the colon will be removed from the tag and sent to the function as the value of the numbered parameter. This is designed to be used with templates for devices that have a single function that can select from a large number of choices.

For example, if the System Functions XML contains the following function declaration:

```
<function name="Launch by Name" export="Launch" repeatrate="0" buttontag="App:
↪#PARAMETER1">
  <parameter name="Channel Name" type="string" />
</function>
```

Then a button tagged with `App:Netflix` will call the driver function `Launch("Netflix")`, a button tagged with `App:Prime Video` will call the driver function `Launch("Prime Video")`, a button tagged with `App:HBO Max` will call the driver function `Launch("HBO Max")`, etc. These variations are are not declared individually in the *SystemVariables.xml*, the function calls are created as the tags are matched in Integration Designer.

2.4 Templates

Templates designed for Integration Designer 10 come with their buttons already tagged. Drivers that support autoprogramming contain a new XML file, *DeviceDescription.xml*, which specifies the template or templates that should be used for that device. Since it is impossible for the templates to support every feature of every device, they focus on a subset of common features for each category of device. A driver can specify multiple templates that it can be matched with, so by listing them in order of most specific to least specific a driver can increase its chances of autoprogramming at least some functionality.

2.5 Multi-Instance Drivers

Since a single driver can be used to control more than one device, Integration Designer 10 introduces the concept of *Multi-instance Drivers*. A Multi-instance driver appears as more than one device node in the Workspace in Integration Designer. This allows the individual devices to be assigned to separate rooms even if they are controlled by the same driver. This functionality can also be used to expose composite devices that provide more than one type of functionality. For example, an A/V receiver driver can expose an "AM/FM Tuner" source and an "Internet Streaming" source. This will cause a UI to be generated to control both sources.

Multi-zone audio sources can also take advantage of this to split up their outputs. Each output can be separately assigned to different rooms, allowing the volume controls in those rooms to be automatically programmed.

2.5.1 `sourceid`

Key to this capability is the `sourceid` attribute which can be applied to any function or variable with a `buttontag` attribute. If the function or variable has a `sourceid` attribute, it will only be considered for autoprogramming when that particular source is being programmed.

For example, consider an A/V receiver with a *play* command for both its built-in iPod dock and its network streaming source. Since the tag in the template for both of those devices will be `Play`, each of the two functions should also be tagged with `Play`. There will be no conflict when autoprogramming as long as the two commands are tagged with different `sourceid` attributes. When the iPod dock source is added to the Integration Designer program, the autoprogramming routine will run looking for tags with source IDs that match the iPod dock source only, so the correct command will be matched.

Some functions and variables may be a property of the entire device and not just a single source (power, for example). These functions and variables should be declared without any `sourceid` attribute. The autoprogramming logic looks for both items with matching `sourceid` attributes and items with no `sourceid` attributes when programming.

If the `source` element in the *DeviceDescription.xml* file contains a `maxitem` attribute, Integration Designer will allow up to that many devices to be controlled by a single instance of the driver. In this case, each device will be sequentially numbered, starting with 1 for the first device added. The `sourceid` for functions and variables will be a combination of the `sourceid` specified in the `source` element with the unit number appended.

2.5.2 Example: Multiple Unit Driver

This example *DeviceDescription.xml* shows how to declare a driver that supports controlling multiple units from a single driver. In this case, we have a Music Player driver which can control up to sixteen players from a single driver.

DeviceDescription.xml

```
<?xml version="1.0" encoding="utf-8"?>
<device>
  <sources>
    <source maxitems="16" name="Player" sourceid="Player">
      <templates>
        <template name="Music Player"/>
      </templates>
    </source>
  </sources>
</device>
```

SystemVariables.xml

```
<?xml version="1.0" encoding="utf-8"?>
<variables>
  <category name="General">
    <!-- Variable 1 is visible for every device using the driver -->
    <variable name="Variable 1" sysvar="VARIABLE1" type="boolean"/>

    <!-- Variable 2 is visible only when editing the Player1 device -->
    <variable name="Variable 2" sysvar="VARIABLE1" type="boolean" sourceid=
↪"Player1" />
  </category>

  <category sourceid="Player1">
    <!-- All the variables in this category are only visible when editing
↪player 1 -->
  </category>

  <category sourceid="Player2">
    <!-- All the variables in this category are only visible when editing
↪player 2 -->
  </category>
</variables>
```

2.5.3 Example: Multiple Source Driver

This example *DeviceDescription.xml* shows how to declare a driver that supports a device with multiple functions (in this case an A/V receiver with several internal sources). Declaring the internal sources separately in this way allows the appropriate autoprogramming template to be specified for each source, and allows the user of the driver to select only the sources they are interested in.

```
<?xml version="1.0" encoding="utf-8"?>
<device>
  <sources>
    <source name="HD Radio" sourceid="HDRadio">
      <templates>
        <template name="HD Radio"/>
        <template name="Tuner"/>
        <template name="AM/FM Radio"/>
      </templates>
    </source>
    <source name="Tuner" sourceid="Tuner">
      <templates>
        <template name="AM/FM Radio"/>
        <template name="Tuner"/>
      </templates>
    </source>
    <source name="Sirius Radio" sourceid="Sirius">
      <templates>
        <template name="Sirius Radio" />
        <template name="Satellite Radio" />
        <template name="Tuner" />
      </templates>
    </source>
    <source name="iPod" sourceid="iPod">
      <templates>
        <template name="iPod" />
        <template name="Music Player" />
      </templates>
    </source>
    <source name="AirPlay" sourceid="AirPlay">
      <templates>
        <template name="AirPlay" />
        <template name="Music Player" />
      </templates>
    </source>
    <source name="Bluetooth" sourceid="Bluetooth">
      <templates>
        <template name="Bluetooth" />
        <template name="Music Player" />
      </templates>
    </source>
    <source name="USB" sourceid="USB">
      <templates>
        <template name="USB" />
        <template name="Music Player" />
```

(continues on next page)

(continued from previous page)

```
      </templates>
    </source>
    <source name="Net Radio" sourceid="NetRadio">
      <templates>
        <template name="Internet Radio" />
        <template name="Music Player" />
      </templates>
    </source>
    <source name="Pandora" sourceid="Pandora">
      <templates>
        <template name="Pandora" />
        <template name="Internet Radio" />
        <template name="Music Player" />
      </templates>
    </source>
    <source name="Spotify" sourceid="Spotify">
      <templates>
        <template name="Spotify" />
        <template name="Internet Radio" />
        <template name="Music Player" />
      </templates>
    </source>
  </sources>
</device>
```

2.5.4 Example: Multi-Zone Audio

This example *DeviceDescription.xml* shows how to declare a multi-zone audio device that supports multiple outputs. Defining the device this way allows the outputs to be assigned to individual rooms so that the volume commands can be autoprogrammed.

```
<?xml version="1.0" encoding="utf-8"?>
<device>
  <sources>
    <source maxitems="8" name="Output" sourceid="Output">
      <capabilities>
        <audio roomvolumesource="true" />
      </capabilities>
    </source>
  </sources>
</device>
```

2.5.5 Example: Combination Device

This example *DeviceDescription.xml* shows how to declare a driver that supports a device with multiple functions *and* multiple outputs (in this case a four zone A/V receiver with several internal sources). Declaring the internal sources separately in this way allows the appropriate autoprogramming template to be specified for each source, and allows the user of the driver to select only the sources they are interested in. Defining the device this way allows the outputs to be assigned to individual rooms so that the volume commands can be autoprogrammed.

```
<?xml version="1.0" encoding="utf-8"?>
<device>
  <sources>
    <source maxitems="4" name="Zone" sourceid="Zone">
      <capabilities>
        <audio roomvolumesource="true" />
      </capabilities>
    </source>
    <source name="HD Radio" sourceid="HDRadio">
      <templates>
        <template name="HD Radio"/>
        <template name="Tuner"/>
        <template name="AM/FM Radio"/>
      </templates>
    </source>
    <source name="Tuner" sourceid="Tuner">
      <templates>
        <template name="AM/FM Radio"/>
        <template name="Tuner"/>
      </templates>
    </source>
    <source name="Sirius Radio" sourceid="Sirius">
      <templates>
        <template name="Sirius Radio"/>
        <template name="Satellite Radio"/>
        <template name="Tuner"/>
      </templates>
    </source>
    <source name="iPod" sourceid="iPod">
      <templates>
        <template name="iPod"/>
        <template name="Music Player"/>
      </templates>
    </source>
    <source name="AirPlay" sourceid="AirPlay">
      <templates>
        <template name="AirPlay"/>
        <template name="Music Player"/>
      </templates>
    </source>
    <source name="Bluetooth" sourceid="Bluetooth">
      <templates>
        <template name="Bluetooth"/>
```

(continues on next page)

(continued from previous page)

```
        <template name="Music Player"/>
      </templates>
    </source>
    <source name="USB" sourceid="USB">
      <templates>
        <template name="USB"/>
        <template name="Music Player"/>
      </templates>
    </source>
    <source name="Net Radio" sourceid="NetRadio">
      <templates>
        <template name="Internet Radio"/>
        <template name="Music Player"/>
      </templates>
    </source>
    <source name="Pandora" sourceid="Pandora">
      <templates>
        <template name="Pandora"/>
        <template name="Internet Radio"/>
        <template name="Music Player"/>
      </templates>
    </source>
    <source name="Spotify" sourceid="Spotify">
      <templates>
        <template name="Spotify"/>
        <template name="Internet Radio"/>
        <template name="Music Player"/>
      </templates>
    </source>
  </sources>
</device>
```

CHAPTER 3

Scripting Concepts

3.1 Event-Driven Programming

XP drivers are written in an event-driven style. This means that the driver does not implement a continuous loop to poll for changes in the system. Instead, the driver registers a series of callback functions that are called by the XP runtime when an event of interest occurs. For example, all low-level communications processing is handled by the XP runtime. When a completed message is received, the driver-registered callback function is called to process the message.

Callback functions should complete their work as quickly as possible and return control to the XP runtime. In particular, long *`System.Sleep()`* routines should not be used. If long delays are required, the driver should create a *Timer Object*, and perform the additional processing when the timer callback is triggered.

If the driver does need to poll periodically, it should create a *ScheduledEvent Object* and define a callback function that is called on the desired schedule.

3.2 Debugging

The primary tool for debugging is the *`System.Print()`* API, coupled with the *TraceViewer* tool. If any debugging statements are left in the released version of the driver, it is good practice to use a hidden configuration option to disable them. This prevents the extra processing work of generating the messages on end-user systems that have no way of logging or displaying the messages. Refer to the sample drivers for details on this technique.

Drivers targeting Integration Designer 9.0 and script engine 11 or higher can also use the new `System.Log()` API and the XP diagnostic driver.

3.3 Extensibility

The driver architecture is designed to separate the creation of drivers from their use in system files. To enable dealers to extend the functionality of the driver without having access to the driver source code, the driver event system was developed. This allows dealer-created macros to run in response to conditions monitored by the driver. The driver developer should signal an event for each condition it detects that may be a useful extension point.

3.4 Views

System Variables normally provide the same information to every interface in the system. To enable each interface to get their own copy of that information (for example, to have each interface maintain their own copy of a browse list for a media server); the driver system also supports variables with multiple views.

3.4.1 System Variables XML

To create a system variable with multiple views, a percent (%) is added to the end of the `sysvar` tag in the *SystemVariables.xml* file.

```
<variable name="Input Text"
          sysvar="Keypad_InputTextView%"
          type="string"
          sample="12345" />
```

Each two-way enabled device in the system is assigned a view number. The system generates a set of variables appending each with a view number. The above XML example will generate the following variables if the system contains three two-way enabled devices:

```
'Keypad_InputTextView%1'
'Keypad_InputTextView%2'
'Keypad_InputTextView%3'
```

The 'Drivers' tab in Integration Designer will only show the variable once as 'Input Text' in our example. When the variable is placed on a device, ID will automatically use the variable associated with the view assigned to the device.

3.4.2 System Functions XML

It is often the case that when there are view specific system variables in a driver that there are also view specific driver commands. A view-centric driver command is one that contains a hidden parameter of type `deviceid`. All parameters of type `deviceid` are hidden by default.

```
<parameter name="view"
           type="deviceid" />
```

When a driver command contains a `deviceid` type parameter, the system will set the view parameter to the view assigned to the device. This parameter can then be used by the driver to apply the command to the correct context.

3.4.3 RTI Script Reference

A driver which supports views should start by reading the view list from the *Config Object*. The system will add the item `SYSTEM::TwoWayDeviceList` to all drivers. This can be thought of as a global variable available to all drivers.

```
var g_list = Config.Get("SYSTEM::TwoWayDeviceList");
```

The data returned from *`Config.Get()`* will be a space separated list of view numbers.

```
'1 2 4 5 9 11'
```

This can easily be parsed using the split command.

```
var view_array = g_list.split(' ');
```

The driver should use this list to instantiate and initialize view-centric data structures and system variables. Driver commands using the `deviceid` parameter type can be compared against this view list to determine which instance the command will affect.

NEW IN INTEGRATION DESIGNER 9.4: Drivers that need a list of *all* user interfaces, even those that do not support two-way variables, can use the `SYSTEM::UIDeviceList` variable in the same manner as the `SYSTEM::TwoWayDeviceList` variable described above. This variable contains everything in the TwoWayDeviceList as well as devices that can send commands but are not capable of displaying two-way variables. If your driver depends on the presence of this variable, be sure to set `minimumSoftwareVersion` in your *DriverManifest.xml* to 9.4 or higher.

3.5 Item Lists

Item lists allow the driver to provide a variable length array of data such as an artist list in a music server.

3.5.1 System Variables XML

Item lists are defined in the *SystemVariables.xml* file using the type `list`. A list variable can also be used in conjunction with views as described in *Views*.

```
<variable name="Browse List"
          sysvar="MusicBrowseListView%"
          type="list" />
```

3.5.2 System Functions XML

It is often necessary for the driver to provide a `SelectItem` driver command to go along with the item list. The function should include an integer parameter that will be used for the selected item index.

```
<function name="Select Item"
          export="SelectMusicItem"
          repeatrate="0" >
  <parameter name="index"
             type="integer"
             min="0"
             max="99999"
             default="0"
             description="list index" />
</function>
```

If the item list was defined to be view specific, the driver command should include the `deviceid` parameter type as well.

```
<function name="Select Item"
          export="SelectMusicItem"
          repeatrate="0" >
  <parameter name="index"
             type="integer"
             min="0"
             max="99999"
             default="0"
             description="list index" />
  <parameter name="view"
             type="deviceid" />
</function>
```

A driver command associated with an item list will also contain an additional parameter containing the index of the item at the top of the scroll window. This parameter is added to the end of whatever

parameters are defined in the *SystemVariables.xml*. The script for the above function might look something like this:

```
function SelectMusicItem(idx, viewed, windowtop)
{
}
```

3.5.3 RTI Script Reference

The driver doesn't need to worry about how this list is being viewed. It only needs to be concerned with filling the list variable with the correct data. The driver script API provides the SystemVarsList object for this purpose. The *SystemVarsList Object* should be created using the system variable name defined in the *SystemVariables.xml* file.

```
var g_mylist = new SystemVarsList("MusicBrowseList");
```

If the list supports multiple views, a *SystemVarsList Object* should be created for each view defined in the global view list.

```
var g_mylist1 = new SystemVarsList("MusicBrowseList%1");
var g_mylist2 = new SystemVarsList("MusicBrowseList%2");
var g_mylist3 = new SystemVarsList("MusicBrowseList%3");
```

Modifying the contents of a list involves the follow steps:

- Open the list - `g_mylist1.Open();`
- Modify the list - `g_mylist1.Insert('test'); g_mylist1.RemoveAll();`
- Close the list - `g_mylist1.Close();`

Remote devices are not notified of the list changes until the *`SystemVarsList.Close()`* method has been called. This allows the driver to add several rows before notifying the remotes. It is recommended that if the entire list contents are known ahead of time, the driver should open the list, insert all of the rows then close the list. If however the list is being loaded on the fly by communicating with an external device, the driver should open/insert/close the list for each block of data received.

3.5.4 Typical Music Server

A typical music sever will use a single view-based item list for browsing the music database. When a list item is selected, the driver will clear the existing data from the list and proceed to load the list with data corresponding to the selected item. For example, if the list contained the items 'Artists', Albums' & 'Songs' and the item 'Artists' was selected, the list would be cleared and filled with artists.

The driver must provide a 'back' list function that navigates up one level. It is common for the driver to load the item list using cached data. This provides a much quicker interface for the end-user. The driver should use the *`SystemVarsList.SetIndexes()`* method to restore the browse position when implementing the 'back' function.

3.6 Memory Usage

Drivers running under script engine version 13 or below are allowed to allocate 1 MB of memory per driver instance. Starting with script engine version 14, this limit was raised to 3 MB. Most drivers use significantly less than 1 MB, but parsing large XML or JSON responses can require memory above this amount. If your driver handles these types of responses, it is recommended that you set the `minimumRuntimeVersion` attribute of the *`driver`* element in the *DriverManifest.xml* to 14 or higher to ensure that your driver is running on a firmware version that provides enough memory.

CHAPTER 4

Driver Info (Help)

Each driver can contain help text accessed through Integration Designer using the Get Info command. The help text is stored in the driver as a single .RTF file, referenced by the `helpTextStream` attribute in *DriverManifest.xml*.

Note: The RTF viewer in Integration Designer does not currently support images or other objects embedded in the RTF file.

The best tool for creating the help text is the WordPad application included with Microsoft Windows. The text formatting will be identical between WordPad and the help viewer. While help files can also be created with Microsoft Word, the RTF that Word exports is considerably more complicated than that created by WordPad, and may not display correctly in the help viewer. One way to fix this is to load the Word-generated RTF into WordPad and re-save it from there before using it to build a driver.

CHAPTER 5

Tools

5.1 PackageDriver

The PackageDriver tool is a command-line utility used to bundle the various XML, RTF, and JS files that make up the driver into the single .RTIDRIVER file that is loaded by Integration Designer. PackageDriver performs XML Schema validation on all XML files before creating the RTIDRIVER file, and will refuse to create the driver if there are any errors in the input files. The output from PackageDriver will indicate the source of the errors in the input files. PackageDriver does not validate the script source files for the driver; these are copied unmodified into the driver package. Additionally, PackageDriver can also be used to create the unique IDs required for each driver.

5.1.1 Usage

```
PackageDriver [-i] [-m ManifestFile.xml] [-o DriverFile.RTIDRIVER]
```

-i

The `-i` option causes PackageDriver to generate a new driver ID code and print it to the screen. *All other options will be ignored and no driver will be processed with this option.* Use this option when starting driver development to get a new ID for each driver.

-m

The `-m` option specifies the name of the driver manifest file. If not provided, it defaults to `DriverManifest.xml`.

-o

The `-o` option specifies the driver output filename. This option overrides the output filename specified in the driver manifest, if present.

5.2 TraceViewer

TraceViewer is used to collect debug output from drivers running on the XP. Use the Connect option on the File menu to create a TCP connection to the XP. Messages printed by drivers running on the XP (using the `System.Print()` API) will be shown in TraceViewer, along with messages generated by the scripting engine.

Individual messages can be selected and copied to the clipboard, or the entire contents of the log can be saved to a file using the File, Save command.

TraceViewer can also be used to toggle the display of hidden driver configuration sections in Integration Designer, by using the View, Show Driver Debug Options in Integration Designer command.

CHAPTER 6

Driver XML Reference

The XML files in a driver package describe the driver and the information it provides to Integration Designer.

6.1 General XML Concepts

6.1.1 XML File Overview

- *DriverManifest.xml* - This file provides the name and description of the driver, and contains references to all other files that make up the driver. Every driver must have this file.
- *DeviceDescription.xml* - This file provides the information needed for the autoprogramming system, and contains references to all other files that make up the driver. Assigning templates and matching buttons cannot be done without this file.
- *ConfigSettings.xml* - This file provides a list of all of the information that the driver needs in order to customize it to a particular installation. Integration Designer collects the requested information in the Driver Configuration Editor, and the XP makes the answers available through the *Config Object*. This file can be omitted if the driver does not require configuration.
- *SystemVariables.xml* - This file provides the names and types of all system variables provided by the driver. Integration Designer uses this information to allow the dealer to use the variables in their programs. This file can be omitted if the driver does not provide any system variables.
- *SystemFunctions.xml* - This file provides the names and parameter types of each system function implemented by the driver. Integration Designer uses this information to allow the dealer to call the functions exported by the driver, ensuring that the correct type of information is passed to them. This file can be omitted if the driver does not implement any functions.

- *SystemEvents.xml* - This file lists the events that the driver can signal. Integration Designer uses this information to build the event list in the Driver Event editor. This file can be omitted if the driver does not signal any events.

6.1.2 Dynamic Configuration Expressions

Note: New in Integration Designer 9.0

Every XML element noted above as taking a condition attribute can have an expression attached to it that controls whether or not it is visible in Integration Designer. If the expression evaluates to TRUE (non-zero), then the associated element is visible to the user. If it evaluates to FALSE, the element is invisible.

In order for *PackageDriver* to process drivers using expressions, the `minimumSoftwareVersion` attribute of the `driver` element in the *DriverManifest.xml* file MUST be set to 9.0 or higher.

Each expression consists of a mixture of references to configuration variable, operators, and constants.

Configuration Variable

The value of a configuration variable (one declared in *ConfigSettings.xml*) can be referenced by prefacing the variable name with a dollar sign ($).

Constants

Decimal integer constants (positive, 0, or negative) can be entered directly.

Operators

The following operators are valid in expressions:

Operator	Description
>	Test whether the value on the left-hand side of the expression is greater than the one on the right.
>=	Test whether the value on the left-hand side of the expression is greater than or equal to the one on the right.
<	Test whether the value on the left-hand side of the expression is less than the one on the right. *NOTE*: Since < is a reserved character in XML, this must be entered as `<`
<=	Test whether the value on the left-hand side of the expression is less than or equal to the one on the right. *NOTE*: Since < is a reserved character in XML, this must be entered as `<=`
==	Test whether the value on the left-hand side of the expression is equal to the one on the right.
!=	Test whether the value on the left-hand side of the expression is not equal to the one on the right.
and	True only if the value on both the left and right sides are true.
or	True if either the value on the left or right sides is true.
not	False if the value on the right side is true, otherwise true if it is false.
len	Returns the length (in characters) of the configuration variable on the right. Empty configuration variables have a length of 0.
in	Tests whether the integer on the left-hand side of the expression is contained within the list on the right. The configuration variable on the right is assumed to be an `intlist` type.

Operator Precedence

Level	Operators
1	in, not, len
2	<, <=, >, >=
3	==, !=
4	and
5	or

Operator precedence can be overridden by placing sub-expressions in parenthesis.

Examples

Test if a boolean configuration variable is true: `$varname`

Test if a boolean configuration variable is false: `not $varname`

Test if configuration variable is not empty: `len $varname > 0`

Since every non-zero value is considered true, the following simplification is equivalent: `len $varname`

Test if configuration variable is greater than 5: `$varname > 5`

Test if the value 5 is contained in the range specified by a configuration variable (`intlist`): `5 in $varname`

Test if the value 5 is not contained in the range specified by a configuration variable: `not (5 in $varname)`

Test if a configuration variable is between 5 and 10 (inclusive): `$varname >= 5 and $varname <= 10`

Note that due to XML attribute value restrictions, this would have to be written as follows in the XML file: `$varname >= 5 and $varname <= 10`

6.1.3 Dynamic Naming

Note: New in Integration Designer 9.0

Most user-visible strings can substitute the value of a configuration variable for all or part of the string. Attributes supporting dynamic naming are marked with (Dynamic Naming Allowed) in the descriptions above. To use dynamic naming, surround the configuration variable name with %% characters at the point where you want the variable contents to appear.

Please also see the note about double-underscore configuration variable names in the documentation for the `variable` attribute of the *`setting`* element. Drivers that declare large numbers of variables which are only useful to Integration Designer (lighting and matrix switch drivers, for example) should use the double-underscore prefix on those variable names to reduce the amount of useless data sent to the XP processor.

Drivers using the dynamic naming features must set the `minimumSoftwareVersion` attribute of the *`driver`* element in the *DriverManifest.xml* file to 9.0 or higher.

6.2 DriverManifest.xml

```
<driverManifest>
  <driver>
    <processorScript/>
    <resource/>
    ...
  </driver>
 </driverManifest>
```

`/driverManifest`

The root element of this file, `driverManifest`, should contain a single *`driver`* element.

`/driverManifest/driver`

This element describes the driver contained in this driver package.

Attributes

- **id** (*string*) – Unique identifier that Integration Designer uses to identify this driver. Every driver must have a **unique** identifier, so this should be changed if starting a new driver from the code of an existing driver. Integration Designer will only allow the "Upgrade Driver" command to work on two drivers that have the same id. This ID must be a standard Globally Unique Identifier (GUID) in the following format: `{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}`. The `-i` option of the *PackageDriver* tool can be used to generate a new ID.
- **name** (*string*) – Name of the driver. This is what Integration Designer will use as the default name of the driver when it is added to the control processor.
- **author** (*string*) – Name of the company or individual that wrote the driver. This is displayed in the driver's information dialog.
- **copyright** (*string*) – Copyright message for the driver. This is displayed in the driver's information dialog.
- **deviceManufacturer** (*string*) – Manufacturer of the device controlled by the driver. This is displayed in the driver's information dialog.
- **deviceModel** (*string*) – Model name or number of the device controlled by the driver. This is displayed in the driver's information dialog
- **driverVersion** (*string*) – Version number of the driver. This must be specified in the format (major).(minor), where major and minor are both integers. No other text should be put in this field.
- **minimumRuntimeVersion** (*integer*) – Minimum version of the RTI script runtime that will process this script. Set this to the lowest number that includes all of the script features that the driver uses. Consult the script API documentation to determine what version is required for each function. Integration Designer will verify that the firmware in the control processor is equal to or greater than this version before downloading the driver to the processor. This value is a single integer that describes the script engine version, not the specific processor firmware version.
- **minimumSoftwareVersion** (*string*) – Minimum version number of Integration Designer that will load this driver. This must be specified in the format (major).(minor), where major and minor are both numbers, and must be at least 7.0 (the first version of Integration Designer with driver support). This is used to prevent older versions of Integration Designer from loading drivers that use new XML features that they would not understand. All features in this guide are understood by version 7.0 and above unless noted otherwise. **Integration Designer (versions 10.0 and later) will ignore this element**
- **minimumApexVersion** (*string*) – Minimum version number of Integration Designer that will load the Apex compatible features of this driver. This must be specified in the format (major).(minor), where major and minor are both numbers, and must be at least 10.0 (the first Apex enabled version of Integration Designer). This is used to prevent Apex enabled versions of Inte-

gration Designer from loading drivers that use new Apex based XML features that they would not understand

- **processorType** (*string*) – The type of processor targeted by the driver. This must be set to the string XP−8 for the type of drivers described in this document. Future processors compatible with drivers written for the XP-8 will still use XP−8 in this field.
- **configSettingsStream** (*string*) – Name of the XML file containing the configuration settings. This file name should not contain a path, the file must be in the same directory as the manifest file. If the driver contains no configuration settings, this attribute can be omitted from the manifest. *ConfigSettings.xml* describes the format of this file.
- **systemVariablesStream** (*string*) – Name of the XML file containing the system variables provided by the driver. This file name should not contain a path, the file must be in the same directory as the manifest file. If the driver provides no system variables, this attribute can be omitted from the manifest. *SystemVariables.xml* describes the format of this file.
- **systemFunctionsStream** (*string*) – Name of the XML file containing the system functions provided by the driver. This file name should not contain a path, the file must be in the same directory as the manifest file. If the driver provides no system functions, this attribute can be omitted from the manifest. *SystemFunctions.xml* describes the format of this file.
- **systemEventsStream** (*string*) – Name of the XML file containing the system events provided by the driver. This file name should not contain a path, the file must be in the same directory as the manifest file. If the driver provides no system events, this attribute can be omitted from the manifest. *SystemEvents.xml* describes the format of this file.
- **deviceDescriptionStream** (*string*) – **New in Integration Designer 10.0**. Name of the XML file containing the information used by Integration Designer 10 or higher to aid in autoprogramming. This file name should not contain a path, the file must be in the same directory as the manifest file. If the file is omitted then no template will be assigned and no pages will be created when the driver is added to the project. *DeviceDescription.xml* describes the format of this file.
- **helpTextStream** (*string*) – Name of the RTF file containing the help text for the driver, shown in the driver info dialog. This file name should not contain a path, the file must be in the same directory as the manifest file. If the driver provides no help text, this attribute can be omitted from the manifest. *Driver Info (Help)* describes the format of this file.

/driverManifest/driver/processorScript

This element references a JavaScript file for the driver code. Multiple `processorScript` elements can be specified if the code is split between multiple JavaScript source files.

Attributes

- **stream** (*string*) – Name of the JavaScript file. This file name should not contain a path, and must be in the same directory as the manifest file.

/driverManifest/driver/resource

New in Integration Designer 9.0. This element references an arbitrary file that can be accessed using the *System.LoadResource()* API. There can be zero or more resource elements per driver.

Attributes

- **stream** (*string*) – Name of the resource file. This file name should not contain a path, and must be in the same directory as the manifest file.

/driverManifest/driver/entitlement

New in Integration Designer 9.14 and 10.4 This element references one or more special features required by the driver.

Attributes

- **name** (*string*) – Name of the requested entitlement.

The following entitlement values are valid:

- kRS485Control : Use the processor's control port for general-purpose RS-485. **This DISABLES normal use of the control port!** DO NOT use this unless the driver and device are both specifically designed to work with the control port pinout. If there is any doubt, you probably do not want to set this entitlement.

6.3 DeviceDescription.xml

```
<device>
  <sources>
    <source/>
    ...
  </sources>
</device>
```

/device

The root element of this file. It must contain at least one *sources* element.

/device/sources

This element describes the sources available in this driver package. The sources element must contain at least one *source* element.

/device/sources/source

This element describes a distinct type of source in the driver. Integration Designer will allow the inclusion of up to maxitems instances of a source into a single project, or a single instance if maxitems is not specified.

Attributes

- **name** (*string*) – (required) Name of the source. This is what Integration Designer will use as the default name of the source when it is added to the project.
- **sourceid** (*string*) – (required) The name that will identify this source in the current project. This name is matched with the `sourceid` attribute in the *ConfigSettings.xml*, *SystemVariables.xml*, *SystemFunctions.xml*, and *SystemEvents.xml* files to determine which items belong to this source.
- **maxitems** (*integer*) – (optional) The number of individual devices that a single instance of the driver can support. This attribute should be omitted if only a single instance is allowed.

If `maxitems` is set to more than one an index value, starting at one and running through `maxitems`, will be appended to the `name` and `sourceid` attributes to create a unique name for each device.

/device/sources/source/templates

This element defines a collection of templates for the driver. It can contain one or more *template* elements. The first matching template is used, so the templates should be listed in order from most specific to most generic.

/device/sources/source/templates/template

Attributes

- **name** (*string*) – (required) Name of the template to be used to create the pages for this device.

Multiple templates may be specified by adding additional `template` elements.

/device/sources/source/capabilities

This optional element allows the device capabilities to be described in detail. Currently, only the `audio` capability is defined.

/device/sources/source/capabilities/audio

This element allows the audio capabilities of the device to be described in detail. Currently, only the `roomvolumesource` capability is defined.

Attributes

- **roomvolumesource** (*boolean*) – Set this capability to true to cause this driver to be defaulted to controlling the volume level when it is added to a room. This should be set to true only on devices **likely** to be the audio source for the room. For example, a multi-room audio system or A/V receiver is likely to be providing audio for a room and so should have this setting set to True. On the other hand, a TV monitor is rarely the volume source for a whole room even if it has speakers, and so this should be set to False (or left out completely) for those devices. Because this will not be true in all cases, the setting can always be changed in Integration Designer when adding the device.

Note: A number of examples of different types of DriverDescriptions are available at *Multi-Instance Drivers*.

6.4 ConfigSettings.xml

```
<configuration>
  <category>
    <setting>
      <choice />
      ...
    </setting>
  ...
  </category>
  ...
</configuration>
```

/configuration

The root element of this file, configuration, contains one or more *category* elements.

/configuration/category

This element defines a collection of settings for the driver. It must contain one or more *setting* elements.

Attributes

- **name** (*string*) – (required) Name of this collection of settings, shown in Integration Designer at the top of the list of settings.
- **description** (*string*) – (optional) Description or explanation of this settings category, shown in the panel at the bottom of the driver settings editor in Integration Designer.
- **hidden** (*boolean*) – (optional) If this setting is set to "true", the settings in this category will not be visible in Integration Designer by default. The *TraceViewer* tool can be used to enable the display of hidden settings. Hidden settings are intended to be used for driver debug configuration.
- **advanced** (*boolean*) – (optional) If this setting is set to "true", the settings in this category will be collapsed by default. If it is not present or set to "false", the category will be expanded by default so that all of its settings are visible. This can be used to hide rarely-used setting to avoid confusing users of the driver.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this category will only be visible if the expression provided evaluates to True. If this attribute is not present, the category will always be visible. See the section *Dynamic Configuration Expressions* for more details.

- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, this category will only be shown when configuring the source matching the `sourceid` value.

/configuration/category/setting

This element describes one configuration setting in the driver settings panel in Integration Designer.

Attributes

- **type** (*string*) – (required) Type of setting, must be one of the following values:
  - `serialport` : This setting will request a serial port from the user. Integration Designer will present a drop-down list of all of the two-way serial ports available on the device, and the corresponding configuration variable will be filled with a handle to the selected serial port. Drivers **must** use a `serialport` configuration item to get a handle to a serial port; they must not attempt to construct serial port handles manually. This allows drivers to automatically support serial ports on slave processors or expansion devices (such as the ESC-2).
  - `integer` : This setting will request a number from the user. Integration Designer will present a text box where the value is entered.
  - `string` : This setting will request a string from the user. Integration Designer will present a text box where the string is entered.
  - `boolean` : This setting will request a boolean (true/false) value from the user. Integration Designer will present a check box that the user can check to indicate true or false.
  - `mcstring` : This setting will request a string from the user, from a list of predefined choices. Integration Designer will present a drop-down list of values that the user must choose from, the user is not allowed to enter any other string. The strings in the list box shown in Integration Designer can be different from the string actually placed in the configuration variable. The values in the drop-down list come from one or more choice elements.
  - `mcinteger` : This setting will request a number from the user, from a list of predefined choices. Integration Designer will present a drop-down list of values that the user must choose from, the user is not allowed to enter any other number. Each number can have a descriptive name that is shown instead of the number, but it is the number that is placed in the configuration variable. The values in the drop-down list come from one or more choice elements.
  - `sysmacro` : This setting will request a system macro number from the user. Integration Designer will present a list of values that the user must choose from. This list will show all of the system macros that have been programmed into the XP. The system macro number will be stored as an integer in the configuration variable and can be passed directly to the *System.RunSystemMacro()* API. **Use of this configuration option is depre-**

cated. A better way to achieve the same result is to have the driver signal a System Event which can be connected to a system macro in Integration Designer.

- `sysvarbool` : This setting will request the user select a boolean system variable from another driver. Integration Designer will present a list of all boolean system variables from every driver in the system. The configuration setting will contain the integer ID of the system variable, which can be passed directly to the *`SystemVars.AddSubscription()`* API.
- `sysvarint` : This setting will request the user select an integer system variable from another driver. Integration Designer will present a list of all integer system variables from every driver in the system. The configuration setting will contain the integer ID of the system variable, which can be passed directly to the *`SystemVars.AddSubscription()`* API.
- `sysvarstring` : This setting will request the user select a string system variable from another driver. Integration Designer will present a list of all string system variables from every driver in the system. The configuration setting will contain the integer ID of the system variable, which can be passed directly to the *`SystemVars.AddSubscription()`* API.
- `intlist` : **New in Integration Designer 9.0** This setting requests a range or a comma-separated list (or any combination of the two) of integers from the user. The primary purpose of this configuration item type is for use with the in operator. Ranges are specified with the lower and upper bounds separated by a : character. The list can contain as many comma-separated individual integers or ranges as necessary.
- `rs485port` : **New in Integration Designer 9.14 and 10.4** This setting requests an RS-485 port from the user. You must have the `kRS485Control` *`entitlement`* in your manifest set to use this option, and your driver and device must both be specifically designed to interface with the RTI control port.

- **`name`** (*`string`*) – (required) Name of this configuration item, as shown in Integration Designer.
- **`variable`** (*`string`*) – (required) Name of the configuration variable created in the configuration database for this setting. This can only contain the a-z, A-Z, 0-9 and _ characters, and cannot have spaces. In Integration Designer 9.0 and higher, if the variable name begins with two underscores, the value of the variable is not sent to the XP processor. This is intended for use with the dynamic configuration and naming features added in version 9.0. If the JavaScript code in driver has no use for the names, use a double-underscore prefix on those configuration variables to reduce the size of the data downloaded to the XP, and prevent naming-only changes from marking the XP data as "needs updating".
- **`description`** (*`string`*) – (optional) Description or explanation of this settings category, shown in the panel at the bottom of the driver settings editor in Integration Designer when this setting is selected.

- **default** (*string*) – (optional) Default value for the setting. This must be in the correct format for the type of the variable. For string values, this can be any valid string. For integer values, this must be a decimal (base 10) integer, or a dollar sign ($) followed by a hexadecimal integer. For multiple-choice types (`mcstring` and `mcinteger`), this should match one of the settings in the list.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this setting will only be visible if the expression provided evaluates to True. If this attribute is not present, the setting will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, this setting will only be shown when configuring the source matching the `sourceid` value.
- **newsourcedefault** (*string*) – (optional) **New in Integration Designer 10.0** Default name of this source when it is added to the Integration Designer file.
- **countsourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, this setting will be hidden in the configuration and the value will automatically be set to the number instances of this source contained in the Integration Designer file. This attribute is only used with sources that have a `maxitems` value set in their *DeviceDescription.xml*.

Note: For examples of how the `newsourcdefault` and `conuntsourceid` attributes are used, see *Multi-Instance Drivers*.

/configuration/category/setting/choice

This element is a child of the *setting* element, and is used to define the available options for the two multiple-choice setting types (`mcstring` and `mcinteger`). It is not valid if the setting is any other type.

Attributes

- **name** (*string*) – (required) Text displayed in the drop-down list in Integration Designer. If no value attribute is supplied, this text is also used to populate the configuration variable.
- **value** (*string*) – (optional) String or integer that is provided in the configuration variable if this item is selected.

6.5 SystemVariables.xml

```
<variables>
  <category>
    <variable />
    ...
  </category>
  ...
</variables>
```

/variables

The root element of this file, variables, contains one or more category elements.

/variables/category

This element defines a collection of system variables for the driver. It must contain one or more *variable* elements.

Attributes

- **name** (*string*) – (required) Name of the variable category, used to build the selection lists in Integration Designer. *Dynamic Naming* is allowed for this item.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this category will only be visible if the expression provided evaluates to True. If this attribute is not present, the category will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, the variables in this category will only be considered for autoprogramming if the source being programmed matches the value of this attribute. See *Multi-Instance Drivers* for more information.

/variables/category/variable

This element describes a single system variable provided by the driver.

Attributes

- **name** (*string*) – (required) Name of the variable displayed in Integration Designer. This string is what the user sees as the variable's name, and may contain embedded spaces and punctuation. *Dynamic Naming* is allowed for this item.
- **sysvar** (*string*) – (required) Name of the variable as written into the variable database by the driver. This must be a valid identifier provided by the script, and is never shown to the user. If the `sysvar` name contains a trailing percent sign (%), a new *view* of the variable is created for every interface in the system. Refer to *Views* for more information on views.
- **type** (*string*) – (required) Data type of the variable, from the following list: `integer`, `string`, `boolean`, `image`, `list`. Refer to the system variable documentation for more information on these types.

- **sample** (*string*) – (optional) Sample value displayed in Integration Designer when this variable is used. This should be a representative value that the variable takes on in actual use. It is used as the text that is displayed in the editor in Integration Designer, and future versions of ID will use it in the emulator as well. For string variables, it is best to specify the longest string that will be encountered by users of the driver, so that text fields can be sized properly. The value of this element should be appropriate for the type of the variable, it will be run through the formatter prior to display in Integration Designer. *Dynamic Naming* is allowed for this item.
- **format** (*string*) – (optional) Format string used to display the variable as text. When this variable is used as button text, the value is formatted according to this string before display. The format element is a string that starts with B, F, I, or L and a colon, followed by the rest of the format string. *Dynamic Naming* is allowed for this item.
  - `B:falsename:truename` : Boolean format string, valid only for boolean variables. If the variable is false, `falsename` will be used as the button text, and if true, `truename` will be used. Empty strings are not supported, so if either value is not used, it should be replaced with a single space (example "B: :True").
  - `F:divisor:%.digitsf` : Fixed-point format string, valid only for integer variables. Since there is no fixed-point variable type, this format string is used to provide decimal values from integer variables. The integer variable is first divided by `divisor` (which must be one of the following: 10, 100, or 1000) and then displayed with `digits` to the right of the decimal point.
  - `I:%0widthd` : Zero-padded format string, valid only for integer variables. *width* must be 2, 3, 4, or 5.
  - `L:value1:name1:value2:name2:...` : List format string, valid only for integer variables. The value of the integer variable is looked up in the list and replaced with the corresponding text. If the value is not in the list, the number will be displayed. Items in the list do not have to be contiguous. Empty strings are not supported, so if a particular value should not be displayed, the name should be set to a single space.

  If a driver provides a fixed set of text responses, it is strongly recommended to implement them as an integer variable and a list format rather than a string variable. This increases network performance as only a single integer is sent, and also allows the dealer to customize or translate the string displayed. It also allows the same variable to be used as text and as the source for a bar graph or image list object.
- **min** (*integer*) – (optional) Integer variables only - specifies the smallest value this variable will ever report. This is used to set the scale if the variable is assigned to a bar graph object.
- **max** (*integer*) – (optional) Integer variables only - specifies the largest value this variable will ever report. This is used to set the scale if the variable is assigned to a bar graph object.

- **width** (*integer*) – (optional) Image variables only - specifies the width of the image in pixels.
- **height** (*integer*) – (optional) Image variables only - specifies the height of the image in pixels.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this variable will only be visible if the expression provided evaluates to True. If this attribute is not present, the variable will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **buttontag** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, Integration Designer will assign this variable to a button with this tag during its autoprogramming process. See *Tags* for more information.
- **tagtype** (*string*) – (optional) **New in Integration Designer 11.0** Boolean variables only - valid values are `reversed`, `visible` or `inactive`. If this attribute is present, the variable will be assigned to the specified state on the buttons with a tag matching the `buttontag` attribute. If a boolean variable has a `buttontag` attribute but is missing the `tagtype` attribute, the system variable will be attached to the "Reversed" state.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, the `buttontag` on this variable will only be considered for autoprogramming if the source being programmed matches the value of this attribute. See *Multi-Instance Drivers* for more information.

6.6 SystemFunctions.xml

```
<functions>
  <category>
    <function>
      <parameter>
        <choice />
        ...
      </parameter>
      ...
    </function>
    ...
  </category>
  ...
</functions>
```

/functions

The root element of this file, functions, contains one or more *category* elements.

/functions/category

This element defines a collection of functions for the driver. It must contain one or more *function* elements.

Attributes

- **name** (*string*) – (required) Name of the function category, used to build the selection lists in Integration Designer. *Dynamic Naming* is allowed for this item.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this category will only be visible if the expression provided evaluates to True. If this attribute is not present, the category will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, the `buttontag` on functions in this category will only be considered for autoprogramming if the source being programmed matches the value of this attribute. See *Multi-Instance Drivers* for more information.

/functions/category/function

This element describes a single system function provided by the driver.

Attributes

- **name** (*string*) – (required) Name of the variable displayed in Integration Designer. This string is what the user sees as the function's name, and may contain embedded spaces and punctuation. *Dynamic Naming* is allowed for this item.
- **export** (*string*) – (required) Name of the function as implemented by the driver. This must be a valid function name in the script. This value must be unique within the driver. If multiple calls to the same function are desired (see the description of hidden parameters for reasons this might be used), a unique export can be created by adding a colon (:) to the end of the export and then adding more text. The colon and the text after are ignored when deciding which function to call; only the text up to the colon is significant.
- **repeatrate** (*integer*) – (optional) Delay, in milliseconds, between multiple calls to the function when it is attached to a button that is held down. A lower number corresponds to a higher repeat rate. Setting this value to 0 disables the repeat function, so only a single function call will be made for each button press. If this value is not specified, it defaults to 200 milliseconds.
- **alias** (*string*) – (optional) **New in Integration Designer 9.0** The alias attribute allows multiple System Function entries to refer back to a single system function. This is used to provide convenience drag-and-drop functions that turn into a more general-purpose function for future maintenance.

  This is related to but distinct from the capability for multiple function entries to reference the same JavaScript export (using a colon in the function name to provide a unique export name). In that case, the user had no indication that multiple commands were actually implemented by the same JavaScript function. The new alias feature causes the function to turn into a different one when it is used. It is a way to provide single-click access to common combinations of parameters for a function. The parameters provided by the

alias must match in type and order to those taken by the function being aliased, although they can differ in whether or not they are hidden.

- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this function will only be visible if the expression provided evaluates to True. If this attribute is not present, the function will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **buttontag** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, Integration Designer will assign this command to a button with this tag during its autoprogramming process. See *Tags* for more information.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, the `buttontag` on this function will only be considered for autoprogramming if the source being programmed matches the value of this attribute. See *Multi-Instance Drivers* for more information.

/functions/category/function/parameter

Each function can have from 0 to 4 parameters, specified by parameter elements.

Attributes

- **name** (*string*) – (required) Name of the parameter shown in Integration Designer. *Dynamic Naming* is allowed for this item.
- **type** (*string*) – (required) Type of the parameter, from the following list:
  - `integer` - Parameter is a number. Integration Designer provides a text field to enter the number. Functions with at least one integer parameter can also be attached to a slider objects, in which case the value is provided by the position touched by the user. The min and max attributes can also be set, and Integration Designer will require that any number entered falls within this range.
  - `boolean` - Parameter is a boolean. Integration Designer provides a drop-down list with two values. The values default to True and False, but can be renamed using the true and false attributes.
  - `string` - Parameter is a string. Integration Designer provides a text field to enter the string. The length of the string can be limited by providing a value for the `maxlength` attribute.
  - `mcstring` - Parameter is a string, chosen from a list provided by Integration Designer. One or more choice elements provide the items for the list.
  - `mcinteger` - Parameter is a number, chosen from a list provided by Integration Designer. One or more choice elements provide the items for the list.
  - `deviceid` - Address of the interface that called the function. This is a specialized type that is automatically filled in; `deviceid` parameters should always be marked as `hidden`. This type is only used in conjunction with view support, see *Views* for more information on views.

- **min** (*integer*) – (optional) `integer` parameters only. Minimum value that will be accepted for the parameter. If this value is specified, the `max` value must also be specified. If this function is assigned to a graph object, this value will be used to set the minimum scale for the object.
- **max** (*integer*) – (optional) `integer` parameters only. Maximum value that will be accepted for the parameter. If this value is specified, the `min` value must also be specified. If this function is assigned to a graph object, this value will be used to set the maximum scale for the object.
- **increment** (*integer*) – (optional) `integer` parameters only. Step size for the parameter. This value is only used if the function is assigned to a graph object, where it sets the minimum valid change in the value. For example, if this is a parameter to a volume function that expects dB * 10, but the function only accepts values in .5 dB increments (5, 10, 15, 20, etc), set this value to 5 and the graph object will only return values that are multiples of 5.
- **default** (*string*) – (optional) Default value for the parameter. This must be specified if the parameter is hidden.
- **description** (*string*) – (optional) Help text for the parameter. This is used by Integration Designer as a tooltip to explain the parameter.
- **true** (*string*) – (optional) Boolean parameters only. The text that is shown in the parameter chooser for the true value. Defaults to True if not specified. *Dynamic Naming* is allowed for this item.
- **false** (*string*) – (optional) Boolean parameters only. The text that is shown in the parameter chooser for the false value. Defaults to False if not specified. *Dynamic Naming* is allowed for this item.
- **maxlength** (*integer*) – (optional) String parameters only. The maximum length of text that will be allowed for the parameter, in characters.
- **hidden** (*boolean*) – (optional) If "True", indicates that this parameter will not be shown in Integration Designer. Since the parameter is invisible, no means will be provided to set a value so a default value must be provided. Integration Designer reserves space in its UI for each parameter, so hidden parameters should come last to prevent disabled parameters in the middle of the list.

  Hidden parameters are provided to allow multiple commands to appear in Integration Designer that all call the same script function. The hidden parameter provides a way to send a value to the function to differentiate which action was requested. For example, a single SetPower() function can be provided, with a parameter that indicates whether the power should be turned on or off. This parameter can be set as hidden in the SystemFunctions.xml and two "On" and "Off" commands can be provided, allowing the user to drag and drop them directly without manually specifying the parameter.

  The hidden parameter capability also makes it easy to convert a one-way RS-232 library to two-way driver functions. A single SendCommand() function can

be provided in the driver script, which takes a single string parameter. The SystemFunctions.xml can list each function individually, with a hidden parameter containing the RS-232 data, and all functions calling the same SendCommand() function.

/functions/category/function/parameter/choice

The choice element is a child of the parameter element, and is used to define the available options for the two multiple-choice parameter types (`mcstring` and `mcinteger`).

Attributes

- **name** (*string*) – (required) Text displayed in the drop-down list in Integration Designer. If no value attribute is supplied, this text is also used to populate the parameter. *Dynamic Naming* is allowed for this item.
- **value** (*string*) – (optional) String or integer that is provided in the parameter if this item is selected.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this choice will only be visible if the expression provided evaluates to True. If this attribute is not present, the choice will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **buttontag** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, Integration Designer will assign this choice to a button with this tag during its autoprogramming process. Note that each choice in a list can have its own `buttontag`. Integration Designer will treat each choice as a separate command when autoprogramming, so that the resulting system function will have any other parameters set to their default values and the multiple-choice parameter set to the value that matches the button tag selected. See *Tags* for more information.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, the `buttontag` on this choice will only be considered for autoprogramming if the source being programmed matches the value of this attribute. See *Multi-Instance Drivers* for more information.

6.7 SystemEvents.xml

```
<events>
  <category>
    <event />
    ...
  </category>
  ...
</events>
```

/events

The root element of this file, events, contains one or more *category* elements.

/events/category

This element defines a collection of events for the driver. It must contain one or more *event* elements.

Attributes

- **name** (*string*) – (required) Name of the event category, used to build the selection list in Integration Designer. *Dynamic Naming* is allowed for this item.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this category will only be visible if the expression provided evaluates to True. If this attribute is not present, the category will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, the events in this category will only be shown when editing the source matching the `sourceid` value.

/events/category/event

This element describes a single system event provided by the driver.

Attributes

- **name** (*string*) – (required) Name of the variable displayed in Integration Designer. This string is what the user sees as the event's name, and may contain embedded spaces and punctuation. *Dynamic Naming* is allowed for this item.
- **tag** (*string*) – (required) Name of the event as signaled by the driver. This must be a valid event identifier passed by the script to the *System.SignalEvent()* API.
- **condition** (*string*) – (optional) **New in Integration Designer 9.0** If this attribute is present, this event will only be visible if the expression provided evaluates to True. If this attribute is not present, the event will always be visible. See the section *Dynamic Configuration Expressions* for more details.
- **sourceid** (*string*) – (optional) **New in Integration Designer 10.0** If this attribute is present, this event will only be shown when editing the source matching the `sourceid` value.

CHAPTER 7

RTI Script Reference

The RTI Script API consists of objects that have properties and functions. There are two types of objects: static and dynamic. Static objects are preallocated for each driver and are managed by the RTI Script engine. Dynamic objects are those that are created within the driver using the 'new' operator. You cannot delete dynamic object once they are created. Some dynamic objects allow reconfiguring without re-creating.

- Static Objects : *System*, *Config*, *SystemVars*, *Persistence*, *Crypto*, *Util*
- Dynamic Objects : *Timer*, *ScheduledEvent*, *Serial*, *TCP*, *UDP*, *SystemVarsList*, *Ping*, *SSH*, *MDNS*

Unless otherwise specified, all properties and functions are available in version 1 of the RTI Script runtime engine.

7.1 System Object

The `System` object provides a variety of functions for debugging and data conversion. There is one System object per driver and it is created automatically by the system.

7.1.1 Properties

`System.`**`Version`**

This is the RTI Script runtime version number. This corresponds to the `minimumRuntimeVersion` in the *DriverManifest.xml*.

Note: Due to a bug in version 2.0 of the XP-8 firmware, this property will be reported as '123' for that version only (it should be version 1). All other versions of XP-8 firmware, and all versions for other products, return the correct value.

Type string

Access read-only

Sample:

```
var version = System.Version;
```

`System.`**`OnShutdownFunc`**

If this property is set, the function will be called when XP shuts down the script engine.

Type function

Access read/write

Sample:

```
System.OnShutdownFunc = MyShutdownFunction;
```

`System.`**`IPAddress`**

The current IP address assigned to the XP. This may return `0.0.0.0` if the XP hasn't acquired a network address at the time of the function call. You will need to set a timer that continuously checks this property until the address is valid.

Type string

Access read-only

Sample:

```
var ipaddr = System.IPAddress;
```

New in script engine version 3.

System.**MACAddress**

The MAC address assigned to the XP.

Type string

Access read-only

Sample:

```
var macaddr = System.MACAddress;
```

New in script engine version 3.

System.**LogLevel**

The current LogInfo level. The attribute should be used in conjunction with the *LogInfo()* function. If the *LogInfo()* call requires JavaScript that you do not want to always run, you should check the current LogLevel to prevent unnecessary processing. The valid log levels are 0 - Off, 1 - Low, 2 - Medium, 3 - High.

Type integer

Access read-only

Sample:

```
if (System.LogLevel==3) {
  ... processing
  System.LogInfo(3, "detailed log message");
}
```

New in script engine version 11.

System.**IPNetMask**

The current net mask assigned to the XP. This may return `0.0.0.0` if the XP hasn't acquired a network address at the time of the function call. You will need to set a timer that continuously checks this property until the address is valid.

Type string

Access read-only

Sample:

```
var netmask = System.IPNetMask;
```

New in script engine version 15.

7.1.2 Methods

System.**Print**(*msg*)

This function sends a text message to *TraceViewer* for debugging. **The message may not include the percent sign as part of the message.**

Arguments

- **msg** (string()) – Debug message to print.

Returns **boolean** – true = success

Sample:

```
System.Print("Test Driver: This is a test message.\r\n");
```

System.**PrintMultiline**(*msg*)

This function sends a text message to *TraceViewer* for debugging. Carriage return/line breaks will be honored. **The message may not include the percent sign as part of the message.**

Arguments

- **msg** (string()) – Debug message to print.

Returns **boolean** – true = success

Sample:

```
System.PrintMultiline("Test Driver: This is a test message.\r\nThe
↪output supports multiple lines\r\n");
```

New in script engine version 20.

System.**Sleep**(*time*)

This function stops the driver from executing for a designated amount of time. Any events that occur while the driver is sleeping will be queued. This should be used for small periods (<500ms). Any delay longer than 500ms should use a *Timer Object* instead.

Arguments

- **time** (integer()) – Sleep time in milliseconds.

Returns **boolean** – true = success

Sample:

```
System.Sleep(300); // sleep for 300 milliseconds
```

System.**GetURL**(*url*)

This function retrieves a web page using an HTTP GET command. **There is a 500KB limit of the size of the returned data.** Use the *HTTP Object* to transfer larger files.

> **Warning:** This is a blocking operation and can stall your driver if the DNS or device is slow to respond. Use the *HTTP Object* for a non-blocking connection. This is especially important when communicating with servers on the Internet where factors beyond your control can cause unpredictable slowdowns in your driver.

Arguments

- **url** (string()) – URL of page to retrieve.

Returns **string** – URL page data. The length will be zero on error.

Sample:

```
var page = System.GetURL("http://example.com/mypage.xml");
```

System.**ConvertFromUTF8**(*utf8_string*)

This function converts a UTF-8 encoded string to a Unicode (UTF-16) string.

Arguments

- **utf8_string** (string()) – UTF-8 encoded string to convert.

Returns **string** – String converted to Unicode (UTF-16).

Sample:

```
var unicode_string = System.ConvertFromUTF8(utf8_string);
```

System.**ConvertToUTF8**(*unicode_string*)

This function converts a Unicode (UTF-16) string to a UTF-8 encoded string.

Arguments

- **unicode_string** (string()) – Unicode (UTF-16) encoded string to convert.

Returns **string** – String converted to UTF-8 encoding.

Sample:

```
var utf8_string = System.ConvertToUTF8(unicode_string);
```

System.**RunSystemMacro**(*macro*)

This function will execute a system macro. *Use of this function is not recommended.* A better way to allow users to run macros at arbitrary points is to signal events using the *System.SignalEvent()* function.

Arguments

- **macro** (integer()) – System macro number from XP macro list. Use a sysmacro configuration item in *ConfigSettings.xml* to obtain the macro number.

Returns **boolean** – `true` = success

Sample:

```
System.RunSystemMacro(10);
```

`System.`**`SignalEvent`**`(`*name*`)`

This function will signal a driver event defined in *SystemEvents.xml*.

Note: Macros attached to the event(s) signaled by the driver are run by the first available macro engine, so signalling multiple events in rapid succession may result in the macros running at the same time. This may cause problems if the macros depend on each other and assume they will be run one at a time.

Arguments

- **name** (`string()`) – Event name.

Returns **boolean** – `true` = success

Sample:

```
System.SignalEvent("ZONE1_ON");
```

`System.`**`SetPriority`**`(`*priority*`)`

This function changes the runtime priority of the driver. For lengthy operations like XML parsing or mathematical calculations, set the driver to a priority of 0. When the operation is complete set the priority back to 5. Doing this prevents the driver from degrading the performance of the rest of the system.

Arguments

- **priority** (`integer()`) – New priority level, in range 0 (lowest) - 5 (highest).

Returns **boolean** – `true` = success

Sample:

```
System.SetPriority(0);
```

`System.`**`StartUPnPScan`**`([`*searchTarget*`])`

This function performs a scan of UPnP devices on the network. The scanning functionality is shared by all drivers.

Arguments

- **searchTarget** (`string()`) – (optional) Search target (ST) value to be used in UPnP broadcast. If omitted, `upnp:rootdevice` is used.

Returns **boolean** – `true` = success

Sample:

```
System.StartUPnPScan("roku:ecp");
```

New in script engine version 9.

Changed in script engine version 20: `searchTarget` parameter added

System.**GetLocalTime**()

This function returns the local time in the form of a space separated string.

Returns **string** – Local time, in the following format:

`year month day dayofweek hour minute second millisecond`

`month` = 1 - January ... 12 - December

`dayofweek` = 0 - Sunday ... 6 - Saturday

Sample:

```
var local_time = System.GetLocalTime();
var local_parts = System.GetLocalTime().split(" ");
```

New in script engine version 9.

System.**GetUTCTime**()

This function returns UTC time in the form of a space separated string.

Returns **string** – UTC time, in the following format:

`year month day dayofweek hour minute second millisecond`

`month` = 1 - January ... 12 - December

`dayofweek` = 0 - Sunday ... 6 - Saturday

Sample:

```
var utc_time = System.GetUTCTime();
var utc_parts = System.GetUTCTime().split(" ");
```

New in script engine version 9.

System.**GetLocalTimeInSeconds**()

This function returns local time in seconds since 1970-01-01.

Returns **integer** – Local time in seconds since 1970-01-01

Sample:

```
var local_time_in_seconds = System.GetLocalTimeInSeconds();
```

New in script engine version 9.

System.**GetUTCTimeInSeconds**()

This function returns UTC time in seconds since 1/1/1970.

Returns **integer** – UTC time in seconds since 1970-01-01

Sample:

```
var utc_time_in_seconds = System.GetUTCTimeInSeconds();
```

New in script engine version 9.

System.**GetTickCount**()

This function returns the number of milliseconds that have elapsed since the system was started.

> **Warning:** The return value is a 32 bit integer and as such will wrap around every 49.7 days. If you are comparing the return values from multiple calls to this API, you must be prepared to handle a wraparound condition (the second call returning a lower value than the first one).

Returns **integer** – Number of milliseconds since the system started

Sample:

```
var current_count = System.GetTickCount();
```

New in script engine version 9.

System.**Compress**(*data*)

This function compresses the input data using the ZLIB format.

Arguments

- **data** (string()) – Data to compress.

Returns **string** – Compressed data

Sample:

```
var compressed = System.Compress(uncompressed);
```

New in script engine version 9.

System.**Uncompress**(*data*, *outputSize*)

This function decompresses the input data using the ZLIB format. The outputSize parameter must be large enough to hold to uncompressed data.

Starting with script engine version 12, this function can also uncompress data in GZIP format. The driver must be marked as requiring script engine version 12 or higher if GZIP-formatted data will be passed to this function.

Arguments

- **data** (string()) – Data to compress.

- **outputSize** (integer()) – Size of output buffer.

Returns **string** – Uncompressed data

Sample:

```
var uncompressed = System.Uncompress(compressed, 1024);
```

New in script engine version 9.

Changed in script engine version 12: Added support for decompressing GZIP-formatted data

System.**GetRandomInteger**(*low_range*, *high_range*)

This function returns a random integer between the low and high range inclusively.

Arguments

- **low_range** (integer()) – Lower bound of returned integer
- **high_range** (integer()) – Upper bound of returned integer

Returns **integer** – Random integer in specified range.

Sample:

```
var ran_int = System.GetRandomInteger(1, 125);
```

New in script engine version 10.

System.**LogError**(*msg*)

This function is used to report errors to the admin console. This function should only be used for errors that require action by the integrator. Other information such as warnings and status should be reported using the LogInfo function. All messages reported by LogError will go unfiltered to the admin console so it is important that "non-errors" are not reported. Overuse of this function will cause the integrator to start ignoring "ERROR" messages.

Arguments

- **msg** (string()) – Error message to print.

Returns **boolean** – true = success

Sample:

```
System.LogError("Sample Log Message");
```

Sample Output:

```
001 - 4/5/13 @ 10:26:17.979 AM - (DRVR:ERR) - 255 - DriverName -
Sample Log Message
```

New in script engine version 11.

System.**LogInfo**(*level*, *msg*)

This function is used to report debug information to the admin console. The information should be something that will be useful to help the integrator fix configuration problems and help the

driver developer debug issues in the field. The level indicates that amount of debug data that the driver is outputting. The Low level should be used for infrequent but important information whereas the high level should be used for details such as bytes sent or received.

Arguments

- **level** (integer()) – Output level of message. 1 - Low, 2 - Medium, 3 - High
- **msg** (string()) – Error message to print.

Returns **boolean** – true = success

Sample:

```
System.LogInfo(1, "Sample Log Message");
```

Sample Output:

```
000 - 4/5/13 @ 10:26:17.978 AM - (DRVR:INF) - 255 - DriverName -
Sample Log Message
```

New in script engine version 11.

System.**GetViewName**(*view*)

This function returns the friendly name for the given view.

Arguments

- **view** (integer()) – View parameter.

Returns **string** – Name of view

Sample:

```
var name = System.GetViewName(a_view);
```

New in script engine version 11.

System.**LoadResource**(*resource*)

This function returns a string with the contents of the specified resource listed in the *resource* element of the driver manifest.

Arguments

- **resource** (string()) – Name of the resource (same as the stream name in the driver manifest XML).

Returns **string** – Contents of the specified resource.

Sample:

```
var xmldata = System.LoadResource("my_resource.xml");
```

New in script engine version 11.

`System.`**`Ping`**(*address*)

This function performs an ICMP Ping targeting the device in the `address` parameter.

> **Warning:** This is a blocking operation and can stall your driver if the DNS or device is slow to respond. Use the *Ping Object* for a non-blocking ping.

Arguments

- **`address`** (`string()`) – Host name or IP address. `"192.168.0.1"` or `"example.com"`.

Returns **`boolean`** – `true` = success

Sample:

```
var deviceOnline = System.Ping("192.168.0.100");
```

New in script engine version 18.

Deprecated since script engine version 20: Use the new non-blocking *Ping* object to get more control of the ping operation.

7.2 Config Object

The `Config` object is used to retrieve the configuration values defined in *ConfigSettings.xml*. There is one `Config` object per driver and it is created automatically by the system.

7.2.1 Properties

There are currently no properties defined for this object.

7.2.2 Methods

`Config.`**`Get`**(*name*)

This function returns configuration data.

Arguments

- **`name`** (`string()`) – Name of configuration data from *ConfigSettings.xml*.

Returns **`string`** – Configuration data

Sample:

```
var g_debug = Config.Get("Debug");
```

7.3 Comm Object

The Comm object is the base class of the *Serial*, *TCP*, *UDP*, *MulticastUDP*, *HTTP*, *TCPServer*, *SSH*, and *MDNS* objects. When one of those objects is created it contains both the properties and methods described in the documentation for that object as well as those in this section.

Changed in script engine version 9: **The maximum receive buffer size is 10MB by default**

7.3.1 Properties

Comm.**TxQueueDepth**

This property indicates the current depth of the transmit queue. The transmit queue is only used when the inter-message transmit delay has been set by calling *SetTxInterMsgDelay()*.

Type integer

Access read-only

Sample:

```
var depth = my_comm.TxQueueDepth
```

Comm.**Handle**

This property provides access to the unique handle associated with the Comm object.

Type integer

Access read-only

Sample:

```
var handle = my_comm.Handle;
```

New in script engine version 9.

Comm.**UseHandleInCallbacks**

This property indicates if the handle parameter should be passed along in the OnCommRx and other callbacks.

Type boolean

Access read/write

Sample:

```
var bUsingHandle = my_comm.UseHandleInCallbacks;

my_comm.UseHandleInCallbacks = true;
```

New in script engine version 9.

Comm.**HeartbeatConnectState**

This property provides access to the heartbeat connect state. `true` = connected, `false` = disconnected

Type boolean

Access read-only

Sample:

```
var bConnected = my_comm.HearbeatConnectState;
```

New in script engine version 9.

7.3.2 Methods

Comm.**Write**(*data*[, *rxtimeout*])

This function sends data to the device. The data is sent immediately unless the inter-message transmit delay has been set by calling *SetTxInterMsgDelay()*.

Arguments

- **data** (`string()`) – Data to be sent.
- **rxtimeout** (`integer()`) – Wait for RX for timeout milliseconds. Default is zero.

Returns **boolean** – `true` = success

Sample:

```
my_comm.Write("HELLO\r");
```

Changed in script engine version 4: Added `rxtimeout` parameter

Comm.**Read**(*timeout*)

This function reads data from the Comm object. If framing has been enabled, only a fully framed message will be returned. The function will return when data is present or if the timeout has occurred. This function is used for special situations. It is more common to process device data in the OnCommRx callback.

Arguments

- **timeout** (`integer()`) – Timeout in milliseconds.

Returns **string** – Received data or empty string if no data.

Sample:

```
var rxdata = my_comm.Read(100);
```

Comm.**WaitForRx**(*timeout*)

This function causes the driver to sleep until either receive data is present or the timeout period has occurred. The data will be returned in the OnCommRx function defined when creating the object. This function is often used to prevent the driver from overflowing the device's receive buffer.

Arguments

- **timeout** (integer()) – Timeout in milliseconds.

Returns **boolean** – true = success

Sample:

```
my_comm.WaitForRx(100);
```

Comm.**SetTxInterMsgDelay**(*delay*)

This function enables queuing of transmit activity. The messages are then sent out at the interval defined by the delay parameter.

Arguments

- **delay** (integer()) – Delay in milliseconds.

Returns **boolean** – true = success

Sample:

```
my_comm.SetTxInterMsgDelay(100);
```

Comm.**AddRxFraming**(*"StopChar"*, *stopchar*)

This function adds stop character framing to the Comm object. A message will be returned in the OnCommRX function when the stop character is seen in the data stream. All characters prior to the stop character and including the stop character will be part of the message.

Arguments

- **type** (string()) – "StopChar"
- **stopchar** (string()) – Character to use for message separation.

Returns **boolean** – true = success

Sample:

```
my_comm.AddRxFraming("StopChar", "\r");
```

Comm.**AddRxFraming**(*"StartStopChar"*, *startchar*, *stopchar*)

This function adds stop/start character framing to the Comm object. A message will be returned in the OnCommRX function when the stop character is seen in the data stream. All characters starting with the start character up to and including the stop character will be part of the message.

Arguments

- **type** (string()) – "StartStopChar"

- **startchar** (string()) – Character to use for the start of message.
- **stopchar** (string()) – Character to use for the end of message.

Returns **boolean** – true = success

Sample:

```
my_comm.AddRxFraming("StartStopChar", "$", "\r");
```

Comm.**AddRxFraming**(*"FixedLength"*, *length*)

This function adds fixed length framing to the Comm object. A message will be returned in the OnCommRX function when the designated number of characters has been received.

Arguments

- **type** (string()) – "FixedLength"
- **length** (integer()) – Number of received characters per message.

Returns **boolean** – true = success

Sample:

```
my_comm.AddRxFraming("FixedLength", 10);
```

Comm.**AddRxHTTPFraming**()

This function adds HTTP framing to the Comm object. A message will be returned in the OnCommRX function when valid HTTP message has been received.

Returns **boolean** – true = success

Sample:

```
my_comm.AddRxHTTPFraming();
```

New in script engine version 9.

Comm.**EnableHeartbeat**(*interval*, *sendheartbeatfunc*, *onconnectfunc*, *ondisconnectfunc*)

This function enables the heartbeat feature on this Comm object. The sendheartbeatfunc function will be called once every interval. In this callback the driver should request a small amount of data from the device. When the driver receives the requested data, it calls the *HeartbeatReceived()* method. The onconnectfunc and ondisconnectfunc functions will be called based on driver calls to *HeartbeatReceived()*.

Arguments

- **interval** (integer()) – How often heartbeats are sent in milliseconds.
- **sendheartbeatfunc** (function()) – Callback function *OnSendHeartbeat()* called when a heartbeat message should be sent to the device.
- **onconnectfunc** (function()) – Callback function to call when transitioning to a connected state. Refer to the OnConnect reference in the specific derived class.

- **ondisconnectfunc** (function()) – Callback function to call when transitioning to a disconnected state. Refer to the OnDisconnect reference in the specific derived class.

Returns boolean – true = success

Sample:

```
my_comm.EnableHeartbeat(3000, OnSendHB, OnConnect, OnDisconnect);
```

Comm.**HeartbeatReceived**()

This function should be called when the device responds to a heartbeat request. The calling or not calling of this function determines when the OnConnect and OnDisconnect events will occur.

Returns boolean – true = success

Sample:

```
my_comm.HeartbeatReceived();
```

7.3.3 Callbacks

Comm.**OnSendHeartbeat**([*handle*])

This function is called periodically to indicate when it is time to send another heartbeat message to the connected device.

Arguments

- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Returns none –

Sample:

```
function OnSendHeartbeat([handle])
{
  ...
}
```

7.4 Serial Object

The Serial object is used to communicate with devices using the RS-232 ports of the XP processor and any expansion devices. The driver should allocate an instance of the Serial object for each port in use.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

7.4.1 Properties

`Serial.`**`OnOneWayTxFunc`**

This function is used to hook one-way serial commands destined for the serial port used by the driver. Normally, serial commands on a button are interleaved with driver communications to the device. If the driver wishes to capture these serial commands for further processing, it should set this property to a driver function.

Type function

Access read/write

Sample:

```
var txfunc = my_serial.OnOneWayTxFunc;
```

7.4.2 Constructors

`Serial`(*rxfunc*, *port*, *baudrate*, *databits*, *stopbits*, *parity*, *handshaking*[, *instance*])

This constructor creates a Serial object to be used for RS-232 communications.

Arguments

- **`rxfunc`** (`function()`) – Callback function *OnCommRX()* called when data is received.
- **`port`** (`integer()`) – Serial port to use. This must be a serial port handle received from a `serialport` configuration item in *ConfigSettings.xml*. Do not try to construct this handle manually, as its format is subject to change.
- **`baudrate`** (`integer()`) – `115200`, `57600`, `38400`, `19200`, `9600`, `4800`, `2400`, `1200`, `600`, `300`
- **`databits`** (`integer()`) – 5, 6, 7, 8
- **`stopbits`** (`integer()`) – 1, 2
- **`parity`** (`string()`) – `"None"`, `"Odd"`, `"Even"`, `"Mark"`, `"Space"`
- **`handshaking`** (`string()`) – `"None"`, `"RTS"`, `"DTR"`
- **`instance`** (`object()`) – (optional) Object to pass back in the *OnCommRX()* function. **As of script version 9, the usage of instance has been deprecated in favor of using handle.**

Returns **`boolean`** – `true` = success

Sample:

```
var my_serial = new Serial(OnCommRx, 12345, 19200, 1, 0, "None", "None");
```

Deprecated since script engine version 9: `instance` parameter

7.4.3 Callbacks

`Serial.`**`OnCommRX`**(*data*[, *instance*][, *handle*])

This function is called when received data is ready on the port. The function is defined by the user and passed in to the constructor.

Arguments

- **`data`** (`string()`) – Data received from port.
- **`instance`** (`object()`) – (optional) Object from the `instance` parameter of the constructor. **As of script version 9, the usage of instance has been deprecated in favor of using handle**
- **`handle`** (`integer()`) – (optional) Used to match the *`Handle`* property of the object.

Returns **none** –

Sample:

```
function OnCommRX(data, handle)
{
  ...
}
```

Changed in script engine version 9: Added `handle` parameter

Deprecated since script engine version 9: `instance` parameter

7.5 TCP Object

The TCP object is used to communicate with devices using a network TCP connection. The driver should allocate an instance of the TCP object for each desired usage. In general the TCP object will stay connected to the device. If the connection is dropped the TCP object will automatically try to reconnect. This is true except for cases where the *`TCP.Close()`* method has been called or the constructor has been called without `host` and `port` arguments.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

Changed in script engine version 9: **Receive buffer defaults to 10MB**

7.5.1 Properties

TCP.**OpenState**

Current open state of the TCP object. This only indicates that the TCP object is open. It does not reflect the connection state with an external device. The state will be 0 for closed and 1 for open.

Type integer

Access read-only

Sample:

```
var openstate = my_tcp.OpenState;
```

New in script engine version 2.

TCP.**OnConnectFunc**

Function to call when TCP connection is made.

Type function *OnConnect()*

Access read/write

Sample:

```
my_tcp.OnConnectFunc = OnTCPConnect;
```

New in script engine version 3.

TCP.**OnDisconnectFunc**

Function to call when TCP disconnects.

Type function *OnDisconnect()*

Access read/write

Sample:

```
my_tcp.OnDisconnectFunc = OnTCPDisconnect;
```

New in script engine version 3.

TCP.**ConnectState**

Current connection state of the TCP object. The state will be 0 for Disconnected and 1 for Connected.

Type integer

Access read-only

Sample:

```
var connstate = my_tcp.ConnectState;
```

New in script engine version 3.

7.5.2 Constructors

TCP(*rxfunc*[, *host*][, *port*][, *instance*][, *rx_buffer_size*])

This constructor creates a TCP object. If the `host` and `port` arguments are present, the TCP object will immediately attempt to create a connection. If the `host` and `port` arguments are not included, the driver must use the *TCP.Open()* method.

Arguments

- **rxfunc** (`function()`) – Callback function *OnCommRX()* called when data is received.
- **host** (`string()`) – (optional) Host name or IP address. `"192.168.0.1"` or `"example.com"`.
- **port** (`integer()`) – (optional) Port number
- **instance** (`object()`) – (optional) Object to pass back in `OnCommRx()` function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**
- **rx_buffer_size** (`integer()`) – (optional) Sets the receive and framer buffer max.

Returns **boolean** – `true` = success

Sample:

```
var my_tcp = new TCP(OnCommRx, "www.example.com", 21);

var my_tcp = new TCP(OnCommRx); // call Open method later
```

Changed in script engine version 6: Added `rx_buffer_size` parameter

Deprecated since script engine version 9: `instance` parameter

7.5.3 Methods

TCP.**Open**([*host*][, *port*][, *instance*][, *rx_buffer_size*])

This function will start a connection with the supplied `host` and `port`.

Arguments

- **host** (`string()`) – (optional) Host name or IP address. `"192.168.0.1"` or `"example.com"`.
- **port** (`integer()`) – (optional) Port number
- **instance** (`object()`) – (optional) Object to pass back in RX function.
- **rx_buffer_size** (`integer()`) – (optional) Sets the receive and framer buffer max.

Returns **boolean** – `true` = success

Sample:

```
my_tcp.Open("www.example.com", 21);
```

New in script engine version 2.

Changed in script engine version 6: Added `rx_buffer_size` parameter

TCP.**Close**()

This function will close and stop the connection process for the TCP object.

Returns **boolean** – `true` = success

Sample:

```
my_tcp.Close();
```

New in script engine version 2.

7.5.4 Callbacks

TCP.**OnConnect**([*handle*])

This function is called when a TCP connection is made.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

```
// "handle" requires script engine version 9 or greater.
function OnConnect(handle)
{
  ...
}
```

New in script engine version 3.

TCP.**OnDisconnect**([*handle*])

This function is called when a TCP connection is disconnected.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

```
// "handle" requires script engine version 9 or greater.
function OnDisconnect(handle)
{
```

(continues on next page)

(continued from previous page)

```
  ...
}
```

New in script engine version 3.

TCP.**OnCommRX**(*data*[, *instance*][, *handle*])

This function is called when received data is ready on the port. The function is defined by the user and passed in to the constructor.

Arguments

- **data** (`string()`) – Data received from port.
- **instance** (`object()`) – (optional) Object from new function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**
- **handle** (`integer()`) – (optional) Used to match *Handle* property of object.

Returns **none** –

Sample:

```
function OnCommRX(data, handle);
```

Changed in script engine version 9: Added `handle` parameter

Deprecated since script engine version 9: `instance` parameter

7.6 TCPServer Object

The `TCPServer` object is used to communicate with devices over a TCP connection. The driver should allocate an instance of the `TCPServer` object for each desired usage. The `TCPServer` object allows the driver to listen for TCP connections from clients. The server allows for 25 active TCP clients per object. The port number, RX framing and write restrictions are all indicated by the `ProfileName` parameter.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

New in script engine version 9.

7.6.1 Properties

TCPServer.**Port**

The TCPServer object will choose the port automatically to avoid internal conflicts. Use this property to retrieve the chosen port.

Type integer

Access read-only

Sample:

```
var port = my_tcpserver.Port;
```

TCPServer.**OnClientConnectFunc**

Function to call when client TCP connection is made.

Type function *OnClientConnect()*

Access read/write

Sample:

```
my_server.OnClientConnectFunc = OnClientTCPConnect;
```

TCPServer.**OnClientDisconnectFunc**

Function to call when client disconnects.

Type function *OnClientDisconnect()*

Access read/write

Sample:

```
my_server.OnClientDisconnectFunc = OnClientTCPDisconnect;
```

7.6.2 Constructors

TCPServer(*rxfunc*)

This constructor creates a TCPServer object.

Arguments

- **rxfunc** (function()) – Callback function *OnCommRX()* called when data is received.

Returns **boolean** – true = success

Sample:

```
var my_tcpserver = new TCPServer(OnCommRx);
```

7.6.3 Methods

TCPServer.**Listen**(*profilename*)

This function starts the TCPServer object listening for connections.

Arguments

- **profilename** (string()) – Profile name to use for set up, from the following list:
  - "UPnPEventServer"
  - "GenericServer" **(script engine version 15 or higher)**
- **port** (integer()) – Used with **GenericServer** profile only. Must be greater than 1024 or the call will fail. Port numbers less than 1024 are reserved for system use.

Returns **boolean** – true = success

Sample:

```
my_tcpserver.Listen("UPnPEventServer");

my_tcpserver.Listen("GenericServer", 5150); // Script engine version >=
↪15 only
```

Changed in script engine version 15: Added "GenericServer" profile

TCPServer.**Write**(*channel*, *data*[, *rxtimeout*])

This function sends data to the device using the specified channel.

Arguments

- **channel** (integer()) – Channel number received in the *OnCommRX()* callback.
- **data** (string()) – Data to be sent.
- **rxtimeout** (integer()) – Number of milliseconds to wait for received data. Default is zero.

Returns **boolean** – true = success

Sample:

```
my_comm.Write(324, "HELLO\r");
```

TCPServer.**CloseChannel**(*channel*)

This function will close the TCP connection on the desired channel. Generally, server TCP connections should be closed within 30 seconds after responding to the request to avoid resource depletion.

Arguments

- **channel** (integer()) – Channel number received in the *OnCommRX()* callback.

Returns **boolean** – true = success

Sample:

```
my_tcpserver.CloseChannel(312);
```

7.6.4 Callbacks

TCPServer.**OnClientConnect**(*fromaddr*, *channel*[, *handle*])

Function called when a client TCP connection is made.

Arguments

- **fromaddr** (string()) – IP address of the remote client.
- **channel** (integer()) – Channel identifier of the new client connection. The channel should be used in the *Write()* function when responding and the *CloseChannel()* function when closing the connection.
- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Sample:

```
function OnClientTCPConnect(fromaddr, channel, [handle])
{
  ...
}
```

TCPServer.**OnClientDisconnect**(*channel*[, *handle*])

Function called when a client disconnects.

Arguments

- **channel** (integer()) – Channel that is disconnecting.
- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Sample:

```
function OnClientTCPDisconnect(channel, [handle])
{
  ...
}
```

TCPServer.**OnCommRX**(*channel*, *data*[, *handle*])

This function is called when data is received on the port.

Arguments

- **channel** (`integer()`) – Channel data was received on. The channel should be used in the *`Write()`* function when responding.
- **data** (`string()`) – Data received from port.
- **handle** (`integer()`) – (optional) Used to match *`Handle`* property of object.

Returns **none** –

Sample:

```
function OnCommRX(channel, data, [instance], [handle])
{
  ...
}
```

7.7 HTTP Object

The HTTP object is used to communicate with devices using a network TCP connection. The driver should allocate an instance of the HTTP object for each desired usage. The HTTP object is similar to the TCP object with a few behavioral changes. The HTTP object does not attempt to keep the TCP connection open. It is the responsibility of the user to call the Open method to keep the connection going. The HTTP object is also built on a lightweight socket engine that allows many connections with high performance and error detection. The HTTP object can also be used to make TLS and WebSocket connections by calling the appropriate functions.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

New in script engine version 9.

7.7.1 Properties

HTTP.**OpenState**

Current open state of the HTTP object. This only indicates that the HTTP object is open. It does not reflect the connection state with an external device. The state will be 0 for closed and 1 for open.

Type integer

Access read-only

Sample:

```
var openstate = my_http.OpenState;
```

HTTP.**OnConnectFunc**

Function to call when an HTTP connection is made.

Type function *OnConnect()*

Access read/write

Sample:

```
my_http.OnConnectFunc = OnConnect;
```

HTTP.**OnConnectFailedFunc**

Function to call when an HTTP connection fails. The connection may fail if the host isn't responding or the name resolution has failed.

Type function *OnConnectFailed()*

Access read/write

Sample:

```
my_http.OnConnectFailedFunc = OnConnectFailed;
```

HTTP.**OnDisconnectFunc**

Function to call when the HTTP connection disconnects.

Type function *OnDisconnect()*

Access read/write

Sample:

```
my_http.OnDisconnectFunc = OnDisconnect;
```

HTTP.**OnSSLHandshakeOKFunc**

Function to call when the TLS handshake has completed successfully.

Type function *OnSSLHandshakeOK()*

Access read/write

Sample:

```
my_http.OnSSLHandshakeOKFunc = OnSSLHandshakeOK;
```

New in script engine version 10.

HTTP.**OnSSLHandshakeFailedFunc**

Function to call when the TLS handshake has failed.

Type function *OnSSLHandshakeFailed()*

Access read/write

Sample:

```
my_http.OnSSLHandshakeFailedFunc = OnSSLHandshakeFailed;
```

New in script engine version 10.

HTTP.**ConnectState**

Current connection state of the HTTP object. The state will be 0 for Disconnected and 1 for Connected.

Type integer

Access read-only

Sample:

```
var connstate = my_http.ConnectState;
```

HTTP.**OnWebsocketUpgradeOKFunc**

Function to call when websocket upgrade has succeeded.

Type function *OnWebsocketUpgradeOK()*

Access read/write

Sample:

```
my_http.OnWebsocketUpgradeOKFunc = OnWebsocketUpgradeOK;
```

New in script engine version 20.

HTTP.**OnWebsocketUpgradeFailedFunc**

Function to call when websocket upgrade has failed.

Type function *OnWebsocketUpgradeFailed()*

Access read/write

Sample:

```
my_http.OnWebsocketUpgradeFailedFunc = OnWebsocketUpgradeFailed;
```

New in script engine version 10.

HTTP.**OnWebsocketPingFunc**

Function to call when the websocket receives a ping.

Type function `OnWebsocktPing()`

Access read/write

Sample:

```
my_http.OnWebsocketPingFunc = OnCommWebsocketPing;
```

New in script engine version 24.

HTTP.**SSLHandshakeTimeout**

Number of milliseconds that the remote host has to respond during the SSL handshake before the connection is deemed a failure. If this property isn't set, the system will continue to use a default timeout of 3000ms (consistent with previous firmware versions). If this property is set to 0, the default value is used.

Type integer

Access read/write

Sample:

```
var timeout = my_http.SSLHandshakeTimeout;
my_http.SSLHandshakeTimeout = 5000;
```

New in script engine version 22.

7.7.2 Constructors

HTTP(*rxfunc*, *host*, *port*[, *rx_buffer_size*])

This function creates an HTTP object. If the `host` and `port` arguments are present, the HTTP object will immediately attempt to create a connection. If the `host` and `port` arguments are not included, the driver must use the *Open()* method.

Arguments

- **rxfunc** (`function()`) – Callback function *OnCommRX()* called when data is received.
- **host** (`string()`) – (optional) Host name or IP address. `"192.168.0.1"` or `"example.com"`.
- **port** (`integer()`) – (optional) Port number
- **rx_buffer_size** – (optional) Sets the receive and framer buffer max. The default value is 10MB.

Returns **boolean** – `true` = success

Sample:

```
var my_http = new HTTP(OnCommRx, "www.example.com", 21);

var my_http = new HTTP(OnCommRx); // call Open method later
```

7.7.3 Methods

HTTP.**Open**([*host*, *port*, *rx_buffer_size*])

This function will start a connection with the supplied `host` and `port`.

Arguments

- **Host** (`string()`) – (optional) Host name or IP address. `"192.168.0.1"` or `"example.com"`.
- **port** (`integer()`) – (optional) Port number

- **rx_buffer_size** (integer()) – (optional) Sets the receive and framer buffer max. The default value is 10MB.

Returns **boolean** – true = success

Sample:

```
my_http.Open("www.example.com", 21);
```

HTTP.**Close**()

This function will close and stop the connection process for the HTTP object.

Returns **boolean** – true = success

Sample:

```
my_http.Close();
```

HTTP.**Disconnect**()

This function will stop or end the connection process for the HTTP object. The object will remain open but not connected.

Returns **boolean** – true = success

Sample:

```
my_http.Disconnect();
```

HTTP.**StartSSLHandshake**()

This function starts an SSL session with the server on this connection. If the function returns true, the callback functions should be used to determine success or failure. If the function returns false, the SSL handshake has failed and no callback function will be executed.

Script engine versions 10 - 12 support only SSLv3 connections. Starting with script engine version 13, connections established with this function will also support TLSv1.0 - 1.2. If the driver needs to talk to a server that only supports TLSv1.0 or higher, it should be marked as requiring a minimum script engine version of 13 or higher.

Returns **boolean** – true = success

Sample:

```
my_http.StartSSLHandshake();
```

New in script engine version 10.

Changed in script engine version 13: Added support for TLSv1.0 - 1.2

HTTP.**AddCertificateAuthority**(*certificate*, *mode*)

This function adds to/replaces the existing certificate authority (CA) store. By default, the HTTP object automatically adds the same root CA bundle as the Firefox browser. You may extend or replace this bundle. You can use the *System.LoadResource()* function to import a certificate.

Arguments

- **certificate** (string()) – PEM formatted certificate.
- **mode** (integer()) – 1 = Append, 2 = Replace.

Returns boolean – true = success

Sample:

```
var cert = "...";
my_http.AddCertificateAuthority(cert,1);
```

New in script engine version 18.

HTTP.**LoadClientCertificate**(*certificate*, *key*[, *password*])

This function allows you to load a client certificate.

Arguments

- **certificate** (string()) – PEM formatted certificate.
- **key** (string()) – PEM formatted private key.
- **password** (string()) – (optional) Password to decrypt the private key.

Returns boolean – true = success

Sample:

```
var cert = System.LoadResource("client_cert.pem");
var privatekey = System.LoadResource("client_key.pem");
var rc = g_http.LoadClientCertificate(cert,privatekey,"toomanysecrets");
```

New in script engine version 18.

HTTP.**UpgradeWebsocket**([*path*])

This function upgrades an open HTTP connection to a websocket connection.

Arguments

- **path** (string()) – (optional) Path to invoke the websocket upgrade.

Returns boolean – true = success

Sample:

```
var username = "joe";
my_http.UpgradeWebsocket("?username="+username);
```

New in script engine version 20.

HTTP.**WebsocketAddHeader**(*name*, *value*)

This function adds a custom header to the websocket upgrade message. Call the function once for each extra header value you want to add.

Arguments

- **name** (`string()`) – Header name.
- **value** (`string()`) – Header value.

Returns **boolean** – `true` = success

Sample:

```
my_http.WebsocketAddHeader("Sec-WebSocket-Protocol","v1.api.smartspeaker.
↪audio");
```

New in script engine version 23.

7.7.4 Callbacks

HTTP.**OnConnect**([*handle*])

This function is called when an HTTP connection is made.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

```
function OnConnect(handle)
{
  ...
}
```

HTTP.**OnConnectFailed**([*handle*])

This function is called when an HTTP connection attempt fails.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

```
function OnConnectFailed(handle)
{
  ...
}
```

HTTP.**OnDisconnect**([*handle*])

This function is called when an HTTP connection is disconnected.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

```
function OnDisconnect(handle)
{
  ...
}
```

HTTP.**OnCommRX**(*data*[, *handle*])

This function is called when received data is ready on the port.

Arguments

- **data** (`string()`) – Data received from port.
- **handle** (`integer()`) – (optional) Used to match *Handle* attribute of object.

Returns **none** –

Sample:

```
function OnCommRX(data, handle)
{
  ...
}
```

HTTP.**OnSSLHandshakeOK**([*handle*][, *reason*])

This function is called when a TLS connection is established. It now optionally reports any issues with the certificate, allowing the caller to decide if the connection should continue.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.
- **reason** (`integer()`) – (optional) Returns one of the following values describing the certificate used, from the *Reasons* table.

Returns **none** –

Reasons:

- `0x00` - **X509_OK**: The certificate is valid
- `0x01` - **X509_BADCERT_EXPIRED**: The certificate validity has expired.
- `0x02` - **X509_BADCERT_REVOKED**: The certificate has been revoked (is on a CRL).
- `0x04` - **X509_BADCERT_CN_MISMATCH**: The certificate Common Name (CN) does not match with the expected CN.
- `0x08` - **X509_BADCERT_NOT_TRUSTED**: The certificate is not correctly signed by the trusted CA.
- `0x10` - **X509_BADCRL_NOT_TRUSTED**: The Certificate Revocation List (CRL) is not correctly signed by the trusted CA.

- `0x20` - **X509_BADCRL_EXPIRED**: The CRL is expired.
- `0x40` - **X509_BADCERT_MISSING**: Certificate was missing.
- `0x80` - **X509_BADCERT_SKIP_VERIFY**: Certificate verification was skipped.

Sample

```
function OnSSLHandshakeOK(handle,reason)
{
  System.Print("reason = " + reason);
}
```

New in script engine version 10.

Changed in script engine version 18: The optional `reason` parameter added.

HTTP.**OnSSLHandshakeFailed**([*handle*])

This function is called when a TLS connection attempt fails.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnSSLHandshakeFailed([handle])
{
  ...
}
```

New in script engine version 10.

HTTP.**OnWebsocketUpgradeOK**(*httpCode*[, *handle*])

Function that is called if the websocket upgrade succeeds.

Arguments

- **httpCode** (`integer()`) – HTTP response code from the server. Code 101 indicates a successful upgrade to WebSockets mode.
- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnWebsocketUpgradeOK(httpCode,[handle])
{
  ...
}
```

New in script engine version 20.

HTTP.**OnWebsocketUpgradeFailed**(*httpCode*[, *handle*])

Function that is called if the websocket upgrade fails.

Arguments

- **httpCode** (integer()) – HTTP response code from the server. Code 101 indicates a successful upgrade to WebSockets mode.
- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnWebsocketUpgradeFailed(httpCode,[handle])
{
  ...
}
```

New in script engine version 20.

HTTP.**OnWebsocketPing**([*handle*])

Function to call when the websocket receives a ping.

Arguments

- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnCommWebsocketPing([handle])
{
  ...
}
```

New in script engine version 24.

7.8 UDP Object

The UDP object is used to communicate with devices using network UDP packets. The driver should allocate an instance of the UDP object for each desired usage.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

7.8.1 Properties

There are currently no properties defined for this object.

7.8.2 Constructors

UDP(*rxfunc*, *host*, *port*[, *instance*])

This constructor creates a UDP object.

Arguments

- **rxfunc** (`function()`) – Callback function *OnCommRX()* called when data is received.
- **host** (`string()`) – (optional) Host name or IP address. `"192.168.0.1"` or `"example.com"`.
- **port** (`integer()`) – Port number
- **instance** (`object()`) – (optional) Object to pass back in RX function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**

Returns **boolean** – `true` = success

Sample:

```
var my_udp = new UDP(OnCommRx, "www.example.com", 21);
```

Deprecated since script engine version 9: `instance` parameter

7.8.3 Methods

UDP.**WriteToAddress**(*host*, *port*, *data*)

This function sends a UDP packet to the specified address and port regardless of what the UDP object was initialized with. This function does not change the host and port specified during object creation. This message will be sent from an unbound port.

Arguments

- **host** (`string()`) – Host name or IP address.
- **port** (`integer()`) – Port to send on.
- **data** (`string()`) – Data to be sent.

Returns **boolean** – `true` = success

Sample:

```
my_comm.WriteToAddress("192.168.5.6", 8080, "my data");
```

7.8.4 Callbacks

UDP.**OnCommRX**(*data*[, *instance*][, *handle*])

This function is called when received data is ready on the port.

Arguments

- **data** (`string()`) – Data received from port.
- **instance** (`object()`) – (optional) Object from new function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**
- **handle** (`integer()`) – (optional) Used to match *Handle* property of object.

Returns **none** –

Sample:

```
Function OnCommRX(data, handle);
```

Changed in script engine version 9: Added `handle` parameter

Deprecated since script engine version 9: `instance` parameter

7.9 MulticastUDP Object

The `MulticastUDP` object is used to communicate with devices over a multicast group. The driver should allocate an instance of the `MulticastUDP` object for each desired usage.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

New in script engine version 9.

7.9.1 Properties

`MulticastUDP`.**Enabled**

This property is used to turn on and off the incoming multicast data flow.

Type boolean

Access read/write

Sample:

```
var bEnabled = my_mcastudp.Enabled;

my_mcastudp.Enabled = false;
```

7.9.2 Constructors

MulticastUDP(*rxfunc*, *group*[, *port*])

This constructor creates a MulticastUDP object.

Arguments

- **rxfunc** (function()) – Callback function *OnCommRX()* called when data is received.
- **group** (string()) – Group name. Usually "239.255.255.250" for UPnP. **Group names other than the UPnP group require script engine version 10 or greater**.
- **port** (integer()) – (optional) UDP port. Default is 1900.

Returns **boolean** – true = success

Sample:

```
var my_mcastudp = new MulticastUDP(OnCommRx, "239.255.255.250");

var my_mcastudp = new MulticastUDP(OnCommRx, "239.250.250.250", 7211);
```

Changed in script engine version 10: Added support for group names other than UPnP

7.9.3 Methods

MulticastUDP.**AddFilter**(*filter*)

Filters are used to reduce the amount of incoming multicast data. By default, there are no filters defined and all traffic will flow to the driver. When one or more filters have been added, only the traffic matching a filter will pass to the driver. The typical usage of a filter would be to limit traffic based on UUID(s). A filter is matched when the filter string is found anywhere in the incoming UDP packet.

Arguments

- **filter** (string()) – String to match incoming data against.

Returns **boolean** – true = success

Sample:

```
my_mcastudp.AddFilter("abcd-1234-dcae");
```

MulticastUDP.**RemoveFilter**(*filter*)

This function allow you to remove a previously added filter.

Arguments

- **filter** (string()) – Filter to remove from list.

Returns **boolean** – true = success

Sample:

```
my_mcastudp.RemoveFilter("abcd-1234-dcae");
```

`MulticastUDP.`**`RemoveAllFilters`**`()`

This functions removes all previously added filters.

Returns **`boolean`** – `true` = success

Sample:

```
my_mcastudp.RemoveAllFilters();
```

7.9.4 Callbacks

`MulticastUDP.`**`OnCommRX`**`(`*fromaddr*, *fromport*, *data*[, *handle*]`)`

This function is called when data is received from the port.

Arguments

- **`fromaddr`** (`string()`) – Address of source. Usually in the form "x.x.x.x".
- **`fromport`** (`integer()`) – UDP port of source.
- **`data`** (`string()`) – Data received from port.
- **`handle`** (`integer()`) – (optional) Used to match *`Handle`* property of object.

Returns **`none`** –

Sample:

```
function OnCommRX(fromaddr, data, [handle])
{
  ...
}
```

7.10 Timer Object

The `Timer` object is used for short timeout periods of one minute or less. For longer timeout periods, use the *ScheduledEvent Object*. The driver should allocate one instance of the `Timer` object for each desired usage.

7.10.1 Properties

`Timer.`**`Interval`**

Interval of timer object (in milliseconds).

Type integer

Access read-only

Sample:

```
var interval = my_timer.Interval;
```

`Timer.`**`State`**

Current state of timer object. Idle = 0, Running = 1

Type integer

Access read-only

Sample:

```
var state = my_timer.State;
```

`Timer.`**`Handle`**

This property provides access to the unique handle associated with the timer object.

Type integer

Access read-only

Sample:

```
var handle = my_timer.Handle;
```

`Timer.`**`UseHandleInCallbacks`**

This property indicates if the `handle` parameter should be passed along in the OnTimer callback.

Type boolean

Access read/write

Sample:

```
var bUsingHandle = my_timer.UseHandleInCallbacks;

my_timer.UseHandleInCallbacks = true;
```

New in script engine version 9.

7.10.2 Constructors

Timer()

This constructor creates a timer object. The driver should then use the Start and Stop methods of this object.

Returns **boolean** – `true` = success

Sample:

```
var my_timer = new Timer();
```

7.10.3 Methods

`Timer.`**Start**(*timeoutfunc*, *timeout*[, *instance*])

This function starts the timer using the supplied timeout function and timeout period. The timer must be idle before calling this method.

Arguments

- **timeoutfunc** (`function()`) – Callback function *OnTimer()* called when timeout occurs.
- **timeout** (`integer()`) – Timeout value in milliseconds.
- **instance** (`object()`) – (optional) Object to pass back in timeout function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**

Returns **boolean** – `true` = success

Sample:

```
my_timer.Start(OnTimeout, 1000); // timeout in 1 second
```

Deprecated since script engine version 9: `instance` parameter

`Timer.`**Stop**()

This function stops a timer that is already running. It is permissible to stop a timer that is already stopped.

Returns **boolean** – `true` = success

Sample:

```
my_timer.Stop();
```

7.10.4 Callbacks

`Timer.`**`OnTimer`**`(`[*instance*][, *handle*]`)`

This function is called when a timeout occurs.

Arguments

- **`instance`** (`object()`) – (optional) Object from new function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**
- **`handle`** (`integer()`) – (optional) Used to match *`Handle`* property of object.

Returns **none** –

Sample:

```
function OnTimer(handle);
```

Changed in script engine version 9: Added `handle` parameter

Deprecated since script engine version 9: `instance` parameter

7.11 ScheduledEvent Object

The `ScheduledEvent` object is used for long timeout periods and daily events. For shorter timeout periods, use the *Timer Object*. The driver should allocate one instance of the `ScheduledEvent` object for each desired usage.

7.11.1 Properties

`ScheduledEvent.`**`Enabled`**

Enabled state of scheduled event. Enabled = `true`, Disabled = `false`

Type boolean

Access read-only

Sample:

```
var enabled = my_schedevent.Enabled;
```

`ScheduledEvent.`**`Instance`**

Object to be sent back to function on event. This will be passed to the *`OnScheduledEvent()`* callback. **As of script version 9, the usage of instance has been deprecated in favor of using handle**

Type object

Access read/write

Sample:

```
my_schedevent.Instance = g_object;
```

Deprecated since script engine version 9: Replaced with `Handle`

`ScheduledEvent.`**`Handle`**

This property provides access to the unique handle associated with the scheduled event object.

Type integer

Access read-only

Sample:

```
var handle = my_sevent.Handle;
```

New in script engine version 9.

`ScheduledEvent.`**`UseHandleInCallbacks`**

This property indicates if the `handle` parameter should be passed along in the *`OnScheduledEvent()`* callback.

Type boolean

Access read/write

Sample:

```
var bUsingHandle = my_event.UseHandleInCallbacks;

my_event.UseHandleInCallbacks = true;
```

New in script engine version 9.

7.11.2 Constructors

`ScheduledEvent`(*oneventfunc*, *"Periodic"*, *intervaltype*, *interval*)

This constructor is used to create a periodic event. Periodic events that use the interval type `"Minutes"` are aligned with the system clock. For example, a 15 minutes periodic timer would occur at 12:00, 12:15, 12:30 etc.

Arguments

- **`oneventfunc`** (`function()`) – Callback function *`OnScheduledEvent()`* called when the event occurs.
- **`type`** (`string()`) – `"Periodic"`
- **`intervaltype`** (`string()`) – `"Minutes"`, `"Seconds"`
- **`interval`** (`integer()`) – Time between events in minutes or seconds.

Returns **object** – new ScheduledEvent object or null if unable to create

Sample:

```
var my_schedevent = new ScheduledEvent(OnEvent, "Periodic", "Minutes",
↪15);
```

ScheduledEvent(*oneventfunc*, *"Daily"*, *"TimeOfDay"*, *timeofday*, *daystype*, *daysofweek*)

This constructor is used to create a daily event that occurs at a specified time.

Arguments

- **oneventfunc** (function()) – Callback function *OnScheduledEvent()* called when the event occurs.
- **type** (string()) – "Daily"
- **dailytype** (string()) – "TimeOfDay"
- **timeofday** (string()) – Time of day, in the "HH:MM" 24-hour format. Examples: "01:15", "14:00"
- **daystype** (string()) – One of: "EVEN", "ODD", "BOTH"
- **daysofweek** (string()) – One of: "All", "Sunday", "Monday", ... "Saturday"

Returns **object** – new ScheduledEvent object or null if unable to create

Sample:

```
var my_schedevent = new ScheduledEvent(OnEvent, "Daily", "TimeOfDay",
↪"12:30", "BOTH", "Sunday");
```

ScheduledEvent(*oneventfunc*, *"Daily"*, *"Sunrise"*, *sunrisetype*[, *offset*], *daystype*, *daysofweek*)

This constructor is used to create a daily event that occurs at sunrise.

Arguments

- **oneventfunc** (function()) – Callback function *OnScheduledEvent()* called when the event occurs.
- **type** (string()) – "Daily"
- **dailytype** (string()) – "Sunrise"
- **sunrisetype** (string()) – One of: "On", "Before", "After"
- **offset** (integer()) – Minutes before or after. Not used for "On" type.
- **daystype** (string()) – One of: "EVEN", "ODD", "BOTH"
- **daysofweek** (string()) – One of: "All", "Sunday", "Monday", ... "Saturday"

Returns **object** – new ScheduledEvent object or null if unable to create

Sample:

```
var my_schedevent = new ScheduledEvent(OnEvent, "Daily", "Sunrise", "On",
↪ "BOTH", "Sunday");
```

ScheduledEvent(*oneventfunc*, *"Daily"*, *"Sunset"*, *sunsettype*[, *offset*], *daystype*, *daysofweek*)

This constructor is used to create a daily event that occurs at sunset.

Arguments

- **oneventfunc** (function()) – Callback function *OnScheduledEvent()* called when the event occurs.
- **type** (string()) – "Daily"
- **dailytype** (string()) – "Sunset"
- **sunsettype** (string()) – One of: "On", "Before", "After"
- **offset** (integer()) – Minutes before or after. Not used for "On" type.
- **daystype** (string()) – One of: "EVEN", "ODD", "BOTH"
- **daysofweek** (string()) – One of: "All", "Sunday", "Monday", ... "Saturday"

Returns **object** – new ScheduledEvent object or null if unable to create

Sample:

```
var my_schedevent = new ScheduledEvent(OnEvent, "Daily", "Sunset", "After
↪", 10, "BOTH", "Sunday");
```

7.11.3 Methods

ScheduledEvent.**Reschedule**(...)

This function is used to reconfigure an existing ScheduledEvent object.

Arguments:

Same as constructors.

Returns **boolean** – true = success

Sample:

```
my_schedevent.Reschedule(...);
```

ScheduledEvent.**Disable**()

This function is used to disable the event.

Returns **boolean** – true = success

Sample:

```
my_schedevent.Disable();
```

ScheduledEvent.**Enable**()

This function is used to enable the event.

Returns **boolean** – true = success

Sample:

```
my_schedevent.Enable();
```

7.11.4 Callbacks

ScheduledEvent.**OnScheduledEvent**([*instance*][, *handle*])

This function is called when the event occurs. The function is defined by the user and passed in to the constructor.

Arguments

- **instance** (object()) – (optional) Object from new function. **As of script version 9, the usage of instance has been deprecated in favor of using handle**
- **handle** (integer()) – (optional) Used to match the *Handle* attribute of the object.

Returns **none** –

Sample:

```
function OnScheduledEvent(handle)
{
  ...
}
```

Changed in script engine version 9: Added handle parameter

Deprecated since script engine version 9: instance parameter

7.12 SystemVars Object

The SystemVars object provides access to the system variable database. The variables associated with a particular driver are defined in *SystemVariables.xml*. There is one SystemVars object per driver and it is created automatically by the system.

7.12.1 Properties

`SystemVars.`**`OnSysVarChangeFunc`**

If this property is set, the function will be called when a variable subscribed to by the driver changes. The callback function should contain a single argument used for the system variable ID.

Type function

Access read/write

Sample:

```
SystemVars.OnSysVarChangeFunc = MySysVarChangeFunction;
```

New in script engine version 4.

7.12.2 Methods

`SystemVars.`**`Write`**(*varname*, *data*, *option*)

This function is used to update the value of a system variable. If the variable does not exist prior to the call, a new object will be created.

Arguments

- **`varname`** (`string()`) – Name of system variable from *SystemVariables.xml*
- **`data`** (`varies()`) – Data to write.
- **`option`** (`string()`) – (optional) Write options.
  - `"BOOLEAN"` - Force value to be evaluated as a boolean.
  - `"IMGURL"` - Force value to type 'Image URL'. Use this for cover art.
  - `"ForcePropagate"` - Inform remotes of change even if data didn't. *The only time this option should ever be used is with Image URL variables* when you need the panel to re-fetch the content of the URL even if it doesn't change (for example, a media server that notifies you that the cover art is changed but always sends `http://192.168.0.1/cover.jpg` as the URL instead of using a unique URL for each album). Using this option on integer, boolean, or string variables will increase network traffic and decrease throughput with no benefit, because it forces the system to send an update even if the value didn't change.
  - `"SendImmediate"` - No delay of remote notification. **This option should never be used in production drivers**, as it was only added for debugging purposes. It will lead to degraded performance of the system, especially when ZigBee devices are present, as it disables the normal update-coalescing mechanisms and forces each System Variable update to be sent in its own message instead of bundled with others. This greatly increases the messaging overhead and reduces the throughput of the entire system. The normal

behavior is to collect all System Variable updates that occur in a 200ms period and pack them into a single message.

Returns **boolean** – `true` = success

Sample:

```
SystemVars.Write("MySysVar", false);
```

`SystemVars.`**Read**(*varname*)

This function allows the driver to read from the system variable database. If the object does not exist, the function will return null.

Until script engine version 12, Read() did not return anything for IMGURL variables. Starting at version 12, the URL of the image is returned.

Arguments

- **varname** (`string()`) – Name of system variable from *SystemVariables.xml*.

Returns **object** – Value of system variable or null if variable does not exist.

Sample:

```
var val = SystemVars.Read("MyOtherSysVar");
```

Changed in script engine version 12: URL is returned for IMGURL variables.

`SystemVars.`**AddSubscription**(*id*)

This function allows the driver to be notified when system variables from other drivers in the system have changed. The *OnSysVarChangeFunc()* will be called for variables subscribed to using this function.

This function requires the system variable ID, not the name of the variable being subscribed to. The identifier must be obtained from Integration Designer using the `sysvarbool`, `sysvarint`, or `sysvarstring` configuration settings.

Arguments

- **id** (`integer()`) – ID of the system variable. Use a `sysvarbool`, `sysvarint`, or `sysvarstring` configuration item in *ConfigSettings.xml* to obtain the system variable ID.

Returns **boolean** – `true` = success

Sample:

```
SystemVars.AddSubscription(123);
```

New in script engine version 4.

`SystemVars.`**RemoveSubscription**(*id*)

This function removes a previously added subscription.

Arguments

- **id** (integer()) – ID of the system variable. Use a sysvarbool, sysvarint, or sysvarstring configuration item in *ConfigSettings.xml* to obtain the system variable ID.

Returns **boolean** – true = success

Sample:

```
SystemVars.RemoveSubscription(123);
```

New in script engine version 4.

7.13 SystemVarsList Object

The SystemVarsList object provides functions for Item List processing. The driver should allocate a SystemVarsList object for each desired usage.

7.13.1 Properties

SystemVarsList.**Size**

The number of items in the list.

Type integer

Access read-only

Sample:

```
var size = my_list.Size;
```

SystemVarsList.**MarkedCount**

This property returns the number of marked items in the list. Use the function *GetMarked()* to retrieve the indexes of the marked items.

Type integer

Access read-only

Sample:

```
var count = my_list.MarkedCount;
```

New in script engine version 4.

SystemVarsList.**OnScrollInfoFunc**

Function to call whenever the remote device changes scroll position or the highlight.

Type function *OnScrollInfo()*

Access

read/write

Sample:

```
my_list.OnScrollInfoFunc = MyScrollFunc;
```

New in script engine version 4.

7.13.2 Constructors

SystemVarsList(*varname*)

This constructor creates a new SystemVarsList object.

Arguments

- **varname** (string()) – Name of system variable from *SystemVariables.xml*.

Returns object – New SystemVarsList object or null on failure.

Sample:

```
var my_list = new SystemVarsList ("MySysVar");
```

7.13.3 Methods

SystemVarsList.**Open**()

This function opens a list for modification. The list object must be opened before operations such as *Insert()* and *RemoveAll()* can occur.

Returns boolean – true = success

Sample:

```
my_list.Open();
```

SystemVarsList.**Insert**(*data*)

This function inserts an item at the end of the list. The list must be open. The data type for list elements must be the same for all rows.

Arguments

- **data** (var()) – Data to write. Must be the same type for all rows in list.

Returns boolean – true = success

Sample:

```
my_list.Insert("row data");
```

SystemVarsList.**InsertWithImage**(*data*, *url*)

> **Warning:** Not all controller types support lists with images.
>
> Read the description carefully to learn how to correctly use this function. If you send images to devices that are not capable of displaying them, you will have problems with your driver.

This function inserts an item containing an image at the end of the list. The list must be open. This must only be used with lists where all the items are strings. It is permissible to mix calls to *Insert()* and this function in one list, but all items added with *Insert()* must be strings.

To use this function, the list you are writing to **MUST** be *view based* and you must use the *Config.Get()* function to fetch the value of the built-in SYSTEM::ItemlistImageDevices setting. This will return a space-separated list of the views that support images. For the view numbers that are *not* in this list, you **MUST NOT** use this function, and instead **MUST** use the *Insert()* function to write to those lists. Your driver must be marked with a minimumApexVersion attribute of the *driver* element in the *DriverManifest.xml* file of 11.0 or higher.

Arguments

- **data** (string()) – Data to write.
- **url** (string()) – URL of image for the item.

Returns **boolean** – true = success

Sample:

```
// One-time initialization code

var g_imageDevices = Config.Get("SYSTEM::ItemlistImageDevices").split("
↪");
var g_allDevices = Config.Get("SYSTEM::TwoWayDeviceList").split(" ");

var g_listVar = [];

g_allDevices.forEach(function(view){
    g_listVar[view] = new SystemVarsList("MyListVariableName%" +
↪parseInt(view, 10));
});


// Do this each time you want to modify the list
// Note: The list must be Open before making these calls

g_allDevices.forEach(function(view) {
    if (g_imageDevices.indexOf(view) != -1) {
        g_listVar[view].InsertWithImage("row data", "http://192.168.0.1/
↪cover.jpg");
    } else {
        g_listVar[view].Insert("row data");
```

(continues on next page)

(continued from previous page)

```
    }
});
```

New in script engine version 25.

SystemVarsList.**InsertAt**(*index*, *data*)

This function inserts an item into the list at the specified index. The existing items starting at index will be shifted down. The list must be open. The data type for list elements must be the same for all rows.

Arguments

- **index** (integer()) – Index to place new item at.
- **data** (object()) – Data to write. Must be the same type for all rows in list.

Returns boolean – true = success

Sample:

```
my_list.InsertAt(0, "row data"); // Insert into front of list
```

New in script engine version 4.

SystemVarsList.**ReadAt**(*index*)

This function returns the item data at index.

Arguments

- **index** (integer()) – Index of item to return.

Returns object – Value of system variable or null on failure.

Sample:

```
var data = my_list.ReadAt(10);
```

New in script engine version 4.

SystemVarsList.**ModifyAt**(*index*, *data*)

This function modifies the item at index.

Arguments

- **index** (integer()) – Index of existing item.
- **data** (object()) – Data to write. Must be the same type for all rows in list.

Returns boolean – true = success

Sample:

```
my_list.ModifyAt(3, "new row data"); // Change row 3 data.
```

New in script engine version 4.

SystemVarsList.**RemoveAll**()

This function removes all entries from a list object. The list must be opened first. All marked items will be cleared.

Returns **boolean** – true = success

Sample:

```
my_list.RemoveAll();
```

SystemVarsList.**RemoveAt**(*index*)

This function deletes the item at index. Items after index are shifted up. After this call, the marked list will be empty.

Arguments

- **index** (integer()) – Index of item to delete.

Returns **boolean** – true = success

Sample:

```
my_list.RemoveAt(0); // Remove front item.
```

New in script engine version 4.

SystemVarsList.**SetMarked**(*index*)

This function sets the marked state of the item at index to true. All other list items will have their marked state cleared. The list must be open.

Arguments

- **index** (integer()) – Index of item to set as marked. Use index -1 to clear entire list.

Returns **boolean** – true = success

Sample:

```
my_list.SetMarked(0); // Mark first list item.
```

New in script engine version 4.

SystemVarsList.**AddMarked**(*index*)

This function sets the marked state of the item at index to true. All other list items will remain unchanged. The list must be open.

Arguments

- **index** (integer()) – Index of item to set as marked.

Returns **boolean** – `true` = success

Sample:

```
my_list.AddMarked(0); // Mark first list item.

my_list.AddMarked(10); // Mark item at index 10.
```

New in script engine version 4.

`SystemVarsList.`**RemoveMarked**(*index*)

This function clears the marked state of the item at `index`. All other list items will remain unchanged. The list must be open.

Arguments

- **index** (`integer()`) – Index of item to set as unmarked.

Returns **boolean** – `true` = success

Sample:

```
my_list.RemoveMarked(0); // Unmark first list item.
```

New in script engine version 4.

`SystemVarsList.`**IsMarked**(*index*)

This function returns the marked state of the item at `index`.

Arguments

- **index** (`integer()`) – Index of item to return marked state.

Returns **boolean** – `true` = success

Sample:

```
var marked = my_list.IsMarked(0);
```

New in script engine version 4.

`SystemVarsList.`**GetMarked**(*index*)

This function returns the marked state of the item at `index`. The index is from the group of marked items not the overall list. The index range is from 0 to *MarkedCount* - 1.

Arguments

- **index** (`integer()`) – Index of the marked group list.

Returns **integer** – Mark state of the item, or -1 on failure.

Sample:

```
var firstmarked = my_list.GetMarked(0);
```

New in script engine version 4.

SystemVarsList.**SetIndexes**(*selindex*, *windowtopindex*)

This function allows the driver to change the current selection and window top indexes.

Arguments

- **selindex** (integer()) – Index of new selected item.
- **windowtopindex** (integer()) – Index of window top.

Returns **boolean** – true = success

Sample:

```
my_list.SetIndexes(10, 7);
```

SystemVarsList.**Close**()

This function closes the list and propagates all changes to all devices subscribed to this list.

Returns **boolean** – true = success

Sample:

```
my_list.Close();
```

7.13.4 Callbacks

SystemVarsList.**OnScrollInfo**(*view*, *highlight*, *top*)

This function callback is called whenever the remote device changes scroll position or the highlight.

Arguments

- **view** (integer()) – View number of the device.
- **highlight** (integer()) – Index of the highlighted item.
- **top** (integer()) – Index of the top visible item.

Returns **none** –

Sample:

```
function OnScrollInfo(view, highlight, top)
{
  ...
}
```

New in script engine version 4.

7.14 Persistence Object

The `Persistence` object provides the ability to store data in non-volatile memory. There is one `Persistence` object per driver and it is created automatically by the system.

7.14.1 Properties

There are currently no properties defined for this object.

7.14.2 Methods

`Persistence.`**`Write`**(*key*, *value*)

This function adds or modifies an entry in the driver's persistent store. The data is not written to non-volatile memory at this time.

Arguments

- **key** (`string()`) – Unique name to reference data.
- **value** (`string()`) – Data to store.

Returns **boolean** – `true` = success

Sample:

```
Persistence.Write("MyName", "Joe");
```

`Persistence.`**`Read`**(*key*)

This function reads data from the driver's persistence store. If the data does not exist, `null` is returned.

Arguments

- **key** (`string()`) – Unique name to reference data.

Returns **string** – Persistence data or `null` if data does not exist

Sample:

```
var data = Persistence.Read("MyName");
```

`Persistence.`**`Delete`**([*key*])

This function deletes the data for one entry or the entire persistence store.

Arguments

- **key** (`string()`) – (optional) Unique name to reference data. If no name is specified, all persistence data for driver will be deleted.

Returns **boolean** – `true` = success

Sample:

```
Persistence.Delete("MyName");
```

`Persistence.`**`Save`**`()`

This function writes the current persistence data to non-volatile memory.

Returns **`boolean`** – `true` = success

Sample:

```
Persistence.Save();
```

7.15 Util Object

The `Util` object contains useful functions for manipulating data.

This object contains the properties and functions listed below.

New in script engine version 20.

7.15.1 Properties

There are currently no properties defined for this object.

7.15.2 Methods

`Util.`**`toString`**`(`*data*, *length*, *format*[, *width*]`)`

This function takes a stream of data and formats it in a way that is useful for printing.

Arguments

- **`data`** (`string()`) – data to format.
- **`length`** (`integer()`) – the number of bytes in the data parameter.
- **`format`** (`string()`) – `HEXSTRINGLOWER`, `HEXSTRINGUPPER`, `HEXDUMPLOWER`, `HEXDUMPUPPER`
- **`width`** (`integer()`) – (optional) Width of the hex output when using HEXDUMP mode.

Returns **`string`** – non-empty = success

Sample:

```
var data = "Some data to display, that includes unprintable chars: " +
↪String.fromCharCode(0x00,0x01,0x02,0x03,0x04);
System.Print(Util.toString(data,data.length,"HEXSTRINGUPPER"));
System.Print(Util.toString(data,data.length,"HEXSTRINGLOWER"));
```

(continues on next page)

(continued from previous page)

```
System.Print(Util.toString(data,data.length,"HEXDUMPUPPER"));
System.Print(Util.toString(data,data.length,"HEXDUMPLOWER"));
System.Print(Util.toString(data,data.length,"HEXDUMPLOWER",4));
System.Print(Util.toString(data,data.length,"HEXDUMPLOWER",8));
System.Print(Util.toString(data,data.length,"HEXDUMPLOWER",32));
```

7.16 Crypto Object

The `Crypto` object exposes industry standard cryptographic functions.

This object exposes the properties and functions listed below.

New in script engine version 20.

7.16.1 Properties

`Crypto.`**`Engine`**

This is a string indicating the internal version of the cryptographic library providing the functions to this object.

Type string

Access read-only

Sample:

```
var engine = Crypto.Engine;
```

New in script engine version 24.

7.16.2 Methods

`Crypto.`**`RSAGenerateKey`**`(`*bits*`)`

This function generates an RSA public/private key pair of size *bits*. The resulting pair are concatenated with a double carriage return/line feed. See the example below.

Arguments

- **`bits`** (`integer()`) – The size of the public key in bits.

Returns **`string`** – `key pair` on success.

Sample:

```
var keypair = Crypto.RSAGenerateKey(2048);
var [privatekey,publickey] = keypair.split("\r\n\r\n");
```

Crypto.**RSASign**(*privatekey*, *hashtype*, *data*, *length*)

This function signs a data blob using an RSA private key.

Arguments

- **privatekey** (string()) – PEM format RSA private key.
- **hashtype** (string()) – The type of hash used to create the data. MD5, SHA1, SHA224, SHA256, SHA384, SHA512, RIPEMD160, NONE
- **data** (string()) – Data to sign.
- **length** (integer()) – Length of the *data* provided.

Returns **string** – signature on success.

Sample:

```
var data = String.fromCharCode(0xf9,0xc0,0x43,0x8e,0x6c,0xb8,0x6c,0xd9,
↪0xa0,0xd2,0xa1,0x13,0x2b,0xc6,0x99,0x65,0x08,0x75,0x17,0x94);
var signature = Crypto.RSASign(g_rsaPrivateKey,"SHA1",data,data.length);
```

Crypto.**RSAVerify**(*publickey*, *hashtype*, *hash*, *hashlength*, *signature*, *signaturelength*)

This function verifies a signature of a hash using an RSA public key.

Arguments

- **publickey** (string()) – PEM format RSA public key.
- **hashtype** (string()) – The type of hash used to create the data. MD5, SHA1, SHA224, SHA256, SHA384, SHA512, RIPEMD160, NONE
- **hash** (string()) – Raw hash value.
- **hashlength** (integer()) – *hash* length in bytes.
- **signature** (string()) – The signature to verify.
- **signaturelength** (integer()) – Length of the *signature* provided.

Returns **boolean** – true = success

Sample:

```
var result = Crypto.RSAVerify(g_rsaPublicKey,"SHA1",hash,hash.length,
↪signature,signature.length);
```

Crypto.**Hash**(*digest*, *data*, *length*)

This function calculates the hash of the supplied data.

Arguments

- **digest** (string()) – MD5, SHA1, SHA224, SHA256, SHA384, SHA512, RIPEMD160
- **data** (string()) – Data to be hashed.
- **length** (integer()) – The number of bytes to hash.

Returns **string** – hash on success.

Sample:

```
var data = "Some text to hash";
var hash = Crypto.Hash("SHA1",data,data.length);
```

Crypto.**Base64Encode**(*data*, *length*)

This function base64 encodes the supplied data.

Arguments

- **data** (string()) – Data to be encoded.
- **length** (integer()) – The number of bytes to encode.

Returns **string** – base64 on success.

Sample:

```
var data = "Some text to encode";
var encoded = Crypto.Base64Encode(data,data.length);
```

Crypto.**Base64Decode**(*data*, *length*)

This function decodes a base64 encoded string.

Arguments

- **data** (string()) – Data to be decoded.
- **length** (integer()) – The number of bytes to decode.

Returns **string** – data on success.

Sample:

```
var data = "U29tZSBkYXRhIHRvIGhhc2g=";
var decoded = Crypto.Base64Decode(data,data.length);
```

Crypto.**RSAEncrypt**(*publickey*, *data*, *length*)

This function encrypts a data blob using an RSA public key. The encrypted data can be decrypted with the matching RSA private key. RSA is only able to encrypt data to a maximum length of your key size (2048 bits = 256 bytes) minus padding / header data (11 bytes). This means a 1024 bit key can at most encrypt 128 - 11 = 117 bytes. A 2048 bit key can at most encrypt 256 - 11 = 245 bytes.

Arguments

- **publickey** (string()) – PEM format RSA public key.
- **data** (string()) – Data to encrypt.
- **length** (integer()) – Length of the *data* provided.

Returns **string** – encrypted on success.

Sample:

```
var data = String.fromCharCode(0xf9,0xc0,0x43,0x8e,0x6c,0xb8,0x6c,0xd9,
↪0xa0,0xd2,0xa1,0x13,0x2b,0xc6,0x99,0x65,0x08,0x75,0x17,0x94);
var signature = Crypto.RSAEncrypt(g_rsaPublicKey,data,data.length);
```

Crypto.**RSADecrypt**(*privatekey*, *data*, *length*)

This function decrypts a data blob using an RSA private key. The private key must be paired with the public key used to encrypt the data.

Arguments

- **privatekey** (string()) – PEM format RSA private key.
- **data** (string()) – Data to decrypt.
- **length** (integer()) – Length of the *data* provided.

Returns **string** – decrypted on success.

Sample:

```
var decrypted = Crypto.RSADecrypt(g_rsaPrivateKey,data,data.length);
```

Crypto.**GenerateRandomBitstream**(*bytes*)

This function creates a random sequence of bytes that can be used as a key.

Arguments

- **bytes** (integer()) – Number of random bytes to generate.

Returns **string** – random on success.

Sample:

```
// generate 256bits (32*8) of data
var randomBytes = Crypto.GenerateRandomBitstream(32);
```

Crypto.**AESEncrypt**(*data*, *length*, *key*, *iv*[, *cipher*][, *padding*])

This function encrypts a *data* blob of size *length* using the specified cipher. The encrypted data can be decrypted with *AESDecrypt()*.

Arguments

- **data** (string()) – Data to encrypt.
- **length** (integer()) – Length of the *data* provided.
- **key** (string()) – The key used for encrypting the data.
- **iv** (string()) – The initialization vector for the algorithm.
- **cipher** (integer()) – Optional parameter specifying the cipher. Default cipher if not specified is AES with 128 bit CBC mode. See *Ciphers*.
- **padding** (integer()) – Optional parameter specifying the type of padding to use. Default padding is PKCS7. See *Padding*.

Returns **string** – encrypted on success.

Sample:

```
// AES CBC 128 with one and zeros padding example
var setec = "Setec Astronomy - too many secrets";
var keyData = Crypto.GenerateRandomBitstream(32);
var ivData = Crypto.GenerateRandomBitstream(16);
var result = Crypto.AESEncrypt(setec,setec.length,keyData,ivData,5,1);
```

Ciphers:

- 2 - AES cipher with 128-bit ECB mode.
- 3 - AES cipher with 192-bit ECB mode.
- 4 - AES cipher with 256-bit ECB mode.
- 5 - AES cipher with 128-bit CBC mode.
- 6 - AES cipher with 192-bit CBC mode.
- 7 - AES cipher with 256-bit CBC mode.
- 8 - AES cipher with 128-bit CFB128 mode.
- 9 - AES cipher with 192-bit CFB128 mode.
- 10 - AES cipher with 256-bit CFB128 mode.
- 11 - AES cipher with 128-bit CTR mode.
- 12 - AES cipher with 192-bit CTR mode.
- 13 - AES cipher with 256-bit CTR mode.
- 14 - AES cipher with 128-bit GCM mode.
- 15 - AES cipher with 192-bit GCM mode.
- 16 - AES cipher with 256-bit GCM mode.
- 17 - Camellia cipher with 128-bit ECB mode.
- 18 - Camellia cipher with 192-bit ECB mode.
- 19 - Camellia cipher with 256-bit ECB mode.
- 20 - Camellia cipher with 128-bit CBC mode.
- 21 - Camellia cipher with 192-bit CBC mode.
- 22 - Camellia cipher with 256-bit CBC mode.
- 23 - Camellia cipher with 128-bit CFB128 mode.
- 24 - Camellia cipher with 192-bit CFB128 mode.
- 25 - Camellia cipher with 256-bit CFB128 mode.
- 26 - Camellia cipher with 128-bit CTR mode.
- 27 - Camellia cipher with 192-bit CTR mode.

- 28 - Camellia cipher with 256-bit CTR mode.
- 29 - Camellia cipher with 128-bit GCM mode.
- 30 - Camellia cipher with 192-bit GCM mode.
- 31 - Camellia cipher with 256-bit GCM mode.
- 32 - DES cipher with ECB mode.
- 33 - DES cipher with CBC mode.
- 34 - DES cipher with EDE ECB mode.
- 35 - DES cipher with EDE CBC mode.
- 36 - DES cipher with EDE3 ECB mode.
- 37 - DES cipher with EDE3 CBC mode.
- 38 - Blowfish cipher with ECB mode.
- 39 - Blowfish cipher with CBC mode.
- 40 - Blowfish cipher with CFB64 mode.
- 41 - Blowfish cipher with CTR mode.
- 42 - RC4 cipher with 128-bit mode.
- 43 - AES cipher with 128-bit CCM mode.
- 44 - AES cipher with 192-bit CCM mode.
- 45 - AES cipher with 256-bit CCM mode.
- 46 - Camellia cipher with 128-bit CCM mode.
- 47 - Camellia cipher with 192-bit CCM mode.
- 48 - Camellia cipher with 256-bit CCM mode.
- 49 - Aria cipher with 128-bit key and ECB mode.
- 50 - Aria cipher with 192-bit key and ECB mode.
- 51 - Aria cipher with 256-bit key and ECB mode.
- 52 - Aria cipher with 128-bit key and CBC mode.
- 53 - Aria cipher with 192-bit key and CBC mode.
- 54 - Aria cipher with 256-bit key and CBC mode.
- 55 - Aria cipher with 128-bit key and CFB-128 mode.
- 56 - Aria cipher with 192-bit key and CFB-128 mode.
- 57 - Aria cipher with 256-bit key and CFB-128 mode.
- 58 - Aria cipher with 128-bit key and CTR mode.
- 59 - Aria cipher with 192-bit key and CTR mode.

- `60` - Aria cipher with 256-bit key and CTR mode.
- `61` - Aria cipher with 128-bit key and GCM mode.
- `62` - Aria cipher with 192-bit key and GCM mode.
- `63` - Aria cipher with 256-bit key and GCM mode.
- `64` - Aria cipher with 128-bit key and CCM mode.
- `65` - Aria cipher with 192-bit key and CCM mode.
- `66` - Aria cipher with 256-bit key and CCM mode.
- `67` - AES 128-bit cipher in OFB mode.
- `68` - AES 192-bit cipher in OFB mode.
- `69` - AES 256-bit cipher in OFB mode.
- `70` - AES 128-bit cipher in XTS block mode.
- `71` - AES 256-bit cipher in XTS block mode.
- `72` - ChaCha20 stream cipher.
- `73` - ChaCha20-Poly1305 AEAD cipher.

Padding:

- `0` - PKCS7 padding (default).
- `1` - ISO/IEC 7816-4 padding (one and zeros).
- `2` - ANSI X.923 padding (zeros and length).
- `3` - Zero padding (not reversible).
- `4` - Never pad (full blocks only).

Note: In script engine version 21 this function will accept both integer and string cipher and padding values.

`Crypto.`**`AESDecrypt`**`(`*data*, *length*, *key*, *iv*[, *cipher*][, *padding*]`)`

This function decrypts a *data* blob of size *length* using the specified cipher.

Arguments

- **`data`** (`string()`) – Data to decrypt.
- **`length`** (`integer()`) – Length of the *data* provided.
- **`key`** (`string()`) – The key used for decrypting the data.
- **`iv`** (`string()`) – The initialization vector for the algorithm.

- **cipher** (integer()) – Optional parameter specifying the cipher. Default cipher if not specified is AES with 128 bit CBC mode. See the table of *ciphers* in the description of *AESEncrypt()*.
- **padding** (integer()) – Optional parameter specifying the type of padding to use. Default padding is PKCS7. See the table of *padding* in the description of *AESEncrypt()*.

Returns **string** – decrypted on success.

Sample:

```
// AES CBC 128 with one and zeros padding example
var decrypted = Crypto.AESDecrypt(encrypted,encrypted.length,keyData,
↪ivData,5,1);
```

Note: In script engine version 21 this function will accept both integer and string cipher values.

Crypto.**PBKDF2**(*cipher*, *password*, *iterations*, *salt*)

This function derives a key from a *password*.

Arguments

- **cipher** (integer()) – See the table of *ciphers* in the description of *AESEncrypt()*.
- **password** (string()) – String used to derive the key.
- **iterations** (integer()) – The number of iterations the algorithm should use.
- **salt** (string()) – Data used to salt the key.

Returns **string** – key on success.

Note: In script engine version 21 this function will accept both integer and string cipher values.

Sample:

```
var salt = String.fromCharCode(0x63,0x61,0xb8,0x0e,0x9b,0xdc,0xa6,0x63,
↪0x8d,0x07,0x20,0xf2,0xcc,0x56,0x8f,0xb9);
var cipher = 6;
var password = "12345678";
var iterations = Math.pow(2, 14);
var keybytes = Crypto.PBKDF2(cipher,password,iterations,salt);
```

Crypto.**ChaCha20Poly1305Decrypt**(*key*, *nonce*, *data*[, *tag*])

This function decrypts *data* using a *key*.

Arguments

- **key** (string()) – Symmetric key.

- **nonce** (`string()`) – Arbitrary numerical string that can be used just once in the cryptographic communication. Maximum of 12 bytes.
- **data** (`string()`) – Data to decrypt.
- **tag** (`string()`) – Optional authentication tag.

Returns **string** – `decrypted` on success.

Sample:

```
var decrypted = Crypto.ChaCha20Poly1305Decrypt(key,nonce,encrypted);
```

`Crypto.`**Argon2**(*hashtype*, *password*, *salt*, *opslimit*, *memlimit*, *size*)

This function derives a key from a *password*.

Arguments

- **hashtype** (`string()`) – `ARGON2I`, `ARGON2D`, `ARGON2ID`.
- **password** (`string()`) – Password to derive key from.
- **salt** (`string()`) – Random data to salt the output with.
- **opslimit** (`integer()`) – Number of iterations to perform.
- **memlimit** (`integer()`) – Amount of memory to use in kilobytes.
- **size** (`integer()`) – Number of bytes to return.

Returns **string** – `key` of length `size` on success.

Sample:

```
// stretch "12345678" to a 32 byte key.
var salt = String.fromCharCode(0x77,0x35,0x36,0xdc,0xc3,0x0e,0x2e,0x84,
↪0x7e,0x0e,0x75,0x29,0xe2,0x34,0x60,0xcf);
var opslimit = 4;
var memlimit = 8;
var key = Crypto.Argon2("ARGON2I","12345678",salt,opslimit,memlimit,32);
```

7.17 Ping Object

The Ping object is used to send ICMP ping packets to one or more target devices. The driver should allocate an instance of the Ping object for each group of target devices. As the timeout of an unreceived ICMP packet can take several seconds, a callback feature is used. ICMP packets are sent sequentially, to one device at a time.

New in script engine version 22.

7.17.1 Properties

Ping.**Timeout**

Specifies the maximum number of seconds the system should wait to receive a response from the target device.

Type integer

Access read/write

Sample:

```
my_ping.Timeout = 2;
```

Ping.**Count**

The Ping object supports multiple target devices. This value indicates how many devices are left in the queue.

Type integer

Access read

Sample:

```
var remaining = my_ping.Count;
```

Ping.**UseHandleInCallbacks**

This property indicates if the `handle` parameter should be passed along to the callback functions.

Type boolean

Access read/write

Sample:

```
var bUsingHandle = my_ping.UseHandleInCallbacks;

my_ping.UseHandleInCallbacks = true;
```

Ping.**OnPingOKFunc**

Function to call when the target device has responded to the ICMP packet.

Type function *OnPingOK()*

Access read/write

Sample:

```
my_ping.OnPingOKFunc = OnPingOK;
```

Ping.**OnPingFailedFunc**

Function to call when the target device has not responded to the ICMP packet within the timeout period.

Type function *OnPingFailed()*

Access read/write

Sample:

```
my_ping.OnPingFailedFunc = OnPingFailed;
```

Ping.**OnPingCompleteFunc**

Function to call when all devices in the queue have been pinged.

Type function *OnPingComplete()*

Access read/write

Sample:

```
my_ping.OnPingCompleteFunc = OnPingComplete;
```

7.17.2 Constructors

Ping(*OnPingOK*, *OnPingFailed*, *OnPingComplete*)

This function creates a Ping object.

Arguments

- **OnPingOK** (function()) – Callback function *OnPingOK()* called when a device responds to an ICMP packet.
- **OnPingFailed** (function()) – Callback function *OnPingFailed()* called when a device does not respond to an ICMP packet.
- **OnPingComplete** (function()) – Callback function *OnPingComplete()* called when there are no more addresses to process.

Returns **boolean** – true = success

Sample:

```
var my_ping = new Ping(OnPingOK,OnPingFailed,OnPingComplete);
```

7.17.3 Methods

Ping.**Add**(*address*)

This function adds an address to the queue.

Requires script engine version 20 or greater.

Arguments

- **address** (string()) – Host name or IP address. "192.168.0.1" or "example.com".

Returns **boolean** – true = success

Sample:

```
my_ping.Add("192.168.0.100");
my_ping.Add("example.com");
```

Ping.**Clear**()

This function clears the queue.

Returns boolean – true = success

Sample:

```
my_ping.Clear();
```

Ping.**Start**()

This function begins sending ICMP packets to the addresses in the queue.

Returns boolean – true = success

Sample:

```
my_ping.Start();
```

Ping.**Stop**()

This function halts processing of the queue. The contents of the queue remain the same as before Stop was called.

Returns boolean – true = success

Sample:

```
my_ping.Stop();
```

7.17.4 Callbacks

Ping.**OnPingOK**(*address*, *duration*[, *handle*])

This function is called when a device has responded to the ICMP packet. The function is defined by the user and passed in to the constructor.

Arguments

- **address** (string()) – Address of the device that responded.
- **duration** (integer()) – The number of milliseconds it took for the device to respond.
- **handle** (integer()) – (optional) Used to match Handle attribute of object.

Returns none –

Sample:

```
function OnPingOK(address,duration,handle)
{
  ...
}
```

Ping.**OnPingFailed**(*address*, *reason*[, *handle*])

This function is called when a device has failed to respond to the ICMP packet within the timeout period. The function is defined by the user and passed in to the constructor.

Arguments

- **address** (string()) – Address of the device that responded.
- **reason** (integer()) – The reason the ping failed. See *Reasons*.
- **handle** (integer()) – (optional) Used to match Handle attribute of object.

Returns **none** –

Reasons:

- 0x01 - **PING_REASON_FAIL**: The device failed to respond
- 0x02 - **PING_REASON_ADDRESS**: The device address is invalid.
- 0x03 - **PING_REASON_ICMP_CREATE**: ICMP packet creation issue.
- 0x04 - **PING_REASON_ICMP_SEND**: The system was unable to send the ICMP packet.
- 0x05 - **PING_REASON_ALLOCATE_ERROR**: Memory allocation error.

Sample:

```
function OnPingFailed(address,reason,handle)
{
  ...
}
```

Ping.**OnPingComplete**([*handle*])

This function is called when all addresses in the queue have been processed. The function is defined by the user and passed in to the constructor.

Arguments

- **handle** (integer()) – (optional) Used to match Handle attribute of object.

Returns **none** –

Sample:

```
function OnPingComplete(handle)
{
  ...
}
```

7.18 SSH Object

The SSH object is used to communicate with devices over TCP using the SSH2 protocol. The driver should allocate an instance of the SSH object for each remote service it needs to communicate with.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

New in script engine version 23.

7.18.1 Properties

SSH.**OpenState**

Current open state of the SSH object. This only indicates that the SSH object is open. It does not reflect the connection state with an external device. The state will be 0 for closed and 1 for open.

Type integer

Access read-only

Sample:

```
if (g_ssh.OpenState === 0) {
  // Object is closed
} else {
  // Object is open
}
```

SSH.**OnConnectFunc**

This callback function is invoked when a connection to the remote server is established.

Type function *OnConnect()*

Access read/write

Sample:

```
g_ssh.OnConnectFunc = OnConnect;
```

SSH.**OnConnectFailedFunc**

This callback function is invoked when a connection to the remote server cannot be established.

Type function *OnConnectFailed()*

Access read/write

Sample:

```
g_ssh.OnConnectFailedFunc = OnConnectFailed;
```

SSH.OnDisconnectFunc

This callback function is invoked when the connection with the remote server is severed.

Type function *OnDisconnect()*

Access read/write

Sample:

```
g_ssh.OnDisconnectFunc = OnDisconnect;
```

SSH.OnHandshakeOKFunc

This callback function is invoked when a cryptographically protected channel has been established with the remote server.

Type function *OnHandshakeOK()*

Access read/write

Sample:

```
g_ssh.OnHandshakeOKFunc = OnHandshakeOK;
```

SSH.OnHandshakeFailedFunc

This callback function is invoked if a cryptographically protected channel with the remote server cannot be established.

Type function *OnHandshakeFailed()*

Access read/write

Sample:

```
g_ssh.OnHandshakeFailedFunc = OnHandshakeFailed;
```

SSH.OnAuthenticationOKFunc

This callback function is invoked when authentication succeeds. The SSH object is ready for use after receiving this callback.

Type function *OnAuthenticationOK()*

Access read/write

Sample:

```
g_ssh.OnAuthenticationOKFunc = OnAuthenticationOK;
```

SSH.**OnAuthenticationFailedFunc**

This callback function is invoked when authentication fails.

Type function *OnAuthenticationFailed()*

Access read/write

Sample:

```
g_ssh.OnAuthenticationOKFunc = OnAuthenticationFailed;
```

7.18.2 Constructors

SSH(*rxfunc[, host][, port][, rx_buffer_size]*)

This constructor creates a SSH object. If the `host` argument is present, the SSH object will immediately attempt to create a connection. If the `host` argument is not included, the driver must use the *SSH.Open()* method.

Arguments

- **rxfunc** (`function()`) – Callback function `OnCommRX()` called when data is received.
- **host** (`string()`) – (optional) Remote server host name or IP address.
- **port** (`integer()`) – (optional) Remote port to connect to. If not specified default SSH port 22 is assumed.
- **rx_buffer_size** (`integer()`) – (optional) Sets the receive buffer max. Default size is 8 KB.

Returns **boolean** – `true` = success

Sample:

```
var g_ssh = new SSH(OnCommRx, "raspberrypi.lan");

var g_ssh = new SSH(OnCommRx); // call Open method later

var g_ssh = new SSH(OnCommRx, "192.168.1.101", 22, 16384); // set rx
↪buffer size to 16 KB
```

7.18.3 Methods

SSH.**Open**(*host[, port][, rx_buffer_size]*)

This function will create a connection with the supplied `host` and `port`.

Arguments

- **host** (`string()`) – Host name or IP address. `"192.168.0.1"` or `"example.com"`.

- **port** (integer()) – (optional) Port number. Default is 22.
- **rx_buffer_size** (integer()) – (optional) Sets the receive buffer max. Default size is 8 KB.

Returns boolean – true = success

Sample:

```
g_ssh.Open("raspberrypi.lan", 22, 16384);
```

SSH.**Close**()

This function will close and stop the connection process for the SSH object.

Returns boolean – true = success

Sample:

```
g_ssh.Close();
```

SSH.**Disconnect**()

This function will disconnect from the remote server.

Returns boolean – true = success

Sample:

```
g_ssh.Disconnect();
```

SSH.**StartHandshake**()

This function will start the handshake process with the remote server. Requires the client to already be connected with the server via SSH or Open.

Returns boolean – true = success

Sample:

```
g_ssh.StartHandshake();
```

SSH.**AuthenticatePassword**(*password*)

This function authenticates the user via a password. The server must have indicated its willingness to authenticate in the *OnConnect()* callback.

Arguments

- **password** (string()) – The password to authenticate with.

Returns none –

Sample:

```
g_ssh.AuthenticatePassword("password");
```

SSH.**AuthenticatePublicKey**(*privatekey*[, *privatekeypassword*])

This function authenticates the user via a private key that is paired with the server side public key. The server must have indicated its willingness to authenticate in the *OnConnect()* callback. The private key and password are never sent to the server. Public/private key pairs can be generated using the ssh-keygen tool on Linux systems.

Arguments

- **privatekey** (string()) – RSA private key. A formatted string beginning with '—–BEGIN RSA PRIVATE KEY—–'. This is known as OpenSSH format.
- **privatekeypassword** (string()) – (optional) The password for the private key.

Returns **none** –

Sample:

```
var g_privateKey = System.LoadResource("id_rsa");
g_ssh.AuthenticatePublicKey(g_privateKey,"password");
```

SSH.**AddPromptResponse**(*prompt*, *response*)

This function adds a response for a given prompt. Use this function in conjunction with *AuthenticateKeyboardInteractive*. Most servers will present a prompt of "Password: ".

Arguments

- **prompt** (string()) – The prompt matching the challenge returned by the remote server.
- **response** (string()) – The response that should be sent.

New in script engine version 24.

Returns **none** –

Sample:

```
g_ssh.AddPromptResponse("Password: ","12345");
```

SSH.**AuthenticateKeyboardInteractive**()

This function authenticates via interactive mode. You must register the response for each prompt by calling *AddPromptResponse()*. The server must have indicated its willingness to authenticate in the *OnConnect()* callback.

New in script engine version 24.

Returns **none** –

Sample:

```
g_ssh.AddPromptResponse("Password: ","12345");
g_ssh.AuthenticateKeyboardInteractive();
```

7.18.4 Callbacks

SSH.**OnConnect**([*handle*])

This callback function is invoked when a connection to the remote server is established.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnConnect([handle])
{
  ...
}
```

SSH.**OnConnectFailed**([*handle*])

This callback function is invoked when a connection to the remote server cannot be established.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnConnectFailed([handle])
{
  ...
}
```

SSH.**OnDisconnect**([*handle*])

This callback function is invoked when the connection with the remote server is severed.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnDisconnect([handle])
{
  ...
}
```

SSH.**OnHandshakeOK**(*fingerprint*, *methods*[, *handle*])

This callback function is invoked when a cryptographically protected channel has been established with the remote server.

Arguments

- **fingerprint** (`string()`) – Identifies the server and can be stored and used in subsequent connections to ensure the server hasn't been compromised.
- **method** (`string()`) – Comma separated list that indicates which authentication methods the server will accept. May contain one or more of: `publickey`, `password`, `keyboard-interactive`.
- **handle** (`integer()`) – (optional) Used to match the *`Handle`* property of the object.

Returns **none** –

Sample:

```
function OnHandshakeOK(fingerprint, methods, [handle])
{
  ...
}
```

SSH.**OnHandshakeFailed**([*handle*])

This callback function is invoked if a cryptographically protected channel with the remote server cannot be established.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *`Handle`* property of the object.

Returns **none** –

Sample:

```
function OnHandshakeFailed([handle])
{
  ...
}
```

SSH.**OnAuthenticationOK**([*handle*])

This callback function is invoked when authentication succeeds. The SSH object is ready for use after receiving this callback.

Arguments

- **handle** (`integer()`) – (optional) Used to match the *`Handle`* property of the object.

Returns **none** –

Sample:

```
function OnAuthenticationOK([handle])
{
  ...
}
```

SSH.**OnAuthenticationFailed**([*handle*])

This callback function is invoked when authentication fails.

Arguments

- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnAuthenticationFailed([handle])
{
  ...
}
```

SSH.**OnCommRx**(*data*[, *handle*])

This function is called when received data is ready from the remote server. The function is defined by the user and passed in to the constructor.

Arguments

- **data** (string()) – Data received from port.
- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Returns **none** –

Sample:

```
function OnCommRx(data, handle)
{
  ...
}
```

7.19 MDNS Object

The MDNS object is used to implement discovery of network devices using the multicast DNS (mDNS) protocol. The driver should allocate an instance of the MDNS object for each device type it needs to discover.

This object derives from the *Comm Object* and inherits the properties and methods listed there in addition to those listed below.

New in script engine version 24.

7.19.1 Properties

MDNS.**Timeout**

Sets or gets the timeout value of the MDNS object. This value determines how long the object should actively search for matching devices. The timeout value is in seconds. The default value is 10 seconds.

Type integer

Access read/write

Sample:

```
g_mdns.Timeout = 15;
var timeout = g_mdns.Timeout;
```

7.19.2 Constructors

MDNS(*OnDiscoverFunc*, *OnCompleteFunc*)

This constructor creates a MDNS object.

Arguments

- **OnDiscoverFunc** (function()) – Function to call when a device is discovered. See *OnDiscover()* for the callback function details.
- **OnCompleteFunc** (function()) – Function to call when the *Timeout* duration has expired and no more devices will be reported. See *OnComplete()* for the callback function details.

Returns **boolean** – true = success

Sample:

```
var g_MDNS = new MDNS(OnDiscover, OnComplete);
```

7.19.3 Methods

MDNS.**Discover**(*type*)

This function starts the discovery process. Discovered devices are identified via the *OnDiscover()* callback.

Arguments

- **type** (string()) – String indicating the type of device to search for.

Returns **boolean** – true = success

Sample:

```
g_mdns.Discover("_hap._tcp.local");
```

7.19.4 Callbacks

MDNS.**OnDiscover**(*json*[, *handle*])

This function is called when a device is discovered.

Arguments

- **json** (`string()`) – JSON formatted string describing the device found.
- **handle** (`integer()`) – (optional) Used to match the *Handle* property of the object.

Returns **boolean** – `true` = success

Sample:

```
function OnDiscover(json, handle)
{
  ...
}
```

Sample JSON (formatted for viewing):

```
{
  "Transaction ID": 0,
  "Flags": 33792,
  "Questions": 1,
  "Answer RRs": 1,
  "Authority RRs": 0,
  "Additional RRs": 5,
  "Queries": [
    {
      "Name": "_hap._tcp.local.",
      "Record": "PTR",
      "Class": 1
    }
  ],
  "Answers": [
    {
      "type": "PTR",
      "name": "RainMachine-0094f1._hap._tcp.local.",
      "rclass": 1,
      "ttl": 10,
      "length": 21
    }
  ],
  "Additional": [
    {
      "type": "SRV",
```

(continues on next page)

(continued from previous page)


```
      "name": "RainMachine-0094f1._hap._tcp.local.",
      "service": "RainMachine5.local.",
      "priority": 0,
      "weight": 0,
      "port": 43433
    },
    {
      "type": "TXT",
      "c#": "1",
      "ff": "2",
      "id": "34:E0:C3:B2:A0:D0",
      "md": "SPK5 Pro",
      "pv": "1.1",
      "s#": "1",
      "sf": "1",
      "ci": "28",
      "sh": "1OeFVA=="
    },
    {
      "type": "A",
      "address": "192.168.0.227"
    },
    {
      "type": "AAAA",
      "address": "fe80::abe:77ff:fe00:94f1"
    },
    {
      "type": "AAAA",
      "address": "fd58:7806:b595:10::1"
    }
  ]
}
```


MDNS.**OnComplete**([*handle*])

This function is called once searching has completed. The duration of the search is set via the *Timeout* property.

Arguments

- **handle** (integer()) – (optional) Used to match the *Handle* property of the object.

Returns **boolean** – true = success

Sample:

```
function OnComplete(handle)
{
  ...
}
```

Index

A

Add() (*Ping method*), 109
AddCertificateAuthority() (*HTTP method*), 71
AddFilter() (*MulticastUDP method*), 79
AddMarked() (*SystemVarsList method*), 94
AddPromptResponse() (*SSH method*), 116
AddRxFraming() (*Comm method*), 55, 56
AddRxHTTPFraming() (*Comm method*), 56
AddSubscription() (*SystemVars method*), 89
AESDecrypt() (*Crypto method*), 105
AESEncrypt() (*Crypto method*), 102
Argon2() (*Crypto method*), 107
audio (*device/sources/source/capabilities element*), 29
AuthenticateKeyboardInteractive() (*SSH method*), 116
AuthenticatePassword() (*SSH method*), 115
AuthenticatePublicKey() (*SSH method*), 115

B

Base64Decode() (*Crypto method*), 101
Base64Encode() (*Crypto method*), 101

C

capabilities (*device/sources/source element*), 29
category (*configuration element*), 30
category (*events element*), 40
category (*functions element*), 36
category (*variables element*), 34
ChaCha20Poly1305Decrypt() (*Crypto method*), 106
choice (*configuration/category/setting element*), 33
choice (*functions/category/function/parameter element*), 40
Clear() (*Ping method*), 110
Close() (*HTTP method*), 71
Close() (*SSH method*), 115
Close() (*SystemVarsList method*), 96
Close() (*TCP method*), 62
CloseChannel() (*TCPServer method*), 65
Compress() (*System method*), 49
configuration (*element*), 30
ConnectState (*HTTP attribute*), 68
ConnectState (*TCP attribute*), 60
ConvertFromUTF8() (*System method*), 46
ConvertToUTF8() (*System method*), 46
Count (*Ping attribute*), 108

D

Delete() (*Persistence method*), 97
device (*element*), 28
Disable() (*ScheduledEvent method*), 86
Disconnect() (*HTTP method*), 71
Disconnect() (*SSH method*), 115
Discover() (*MDNS method*), 120
driver (*driverManifest element*), 25
driverManifest (*element*), 25

E

Enable() (*ScheduledEvent method*), 87
Enabled (*MulticastUDP attribute*), 78
Enabled (*ScheduledEvent attribute*), 83
EnableHeartbeat() (*Comm method*), 56
Engine (*Crypto attribute*), 99

entitlement (*driverManifest/driver element*), 28
event (*events/category element*), 41
events (*element*), 40

F

function (*functions/category element*), 37
functions (*element*), 36

G

GenerateRandomBitstream() (*Crypto method*), 102
Get() (*Config method*), 52
GetLocalTime() (*System method*), 48
GetLocalTimeInSeconds() (*System method*), 48
GetMarked() (*SystemVarsList method*), 95
GetRandomInteger() (*System method*), 50
GetTickCount() (*System method*), 49
GetURL() (*System method*), 45
GetUTCTime() (*System method*), 48
GetUTCTimeInSeconds() (*System method*), 48
GetViewName() (*System method*), 51

H

Handle (*Comm attribute*), 53
Handle (*ScheduledEvent attribute*), 84
Handle (*Timer attribute*), 81
Hash() (*Crypto method*), 100
HeartbeatConnectState (*Comm attribute*), 53
HeartbeatReceived() (*Comm method*), 57
HTTP (*module*), 67
HTTP() (*built-in function*), 70

I

-i
PackageDriver command line option, 20
Insert() (*SystemVarsList method*), 91
InsertAt() (*SystemVarsList method*), 93
InsertWithImage() (*SystemVarsList method*), 91
Instance (*ScheduledEvent attribute*), 83
Interval (*Timer attribute*), 81
IPAddress (*System attribute*), 43
IPNetMask (*System attribute*), 44
IsMarked() (*SystemVarsList method*), 95

L

Listen() (*TCPServer method*), 65
LoadClientCertificate() (*HTTP method*), 72
LoadResource() (*System method*), 51
LogError() (*System method*), 50
LogInfo() (*System method*), 50
LogLevel (*System attribute*), 44

M

-m
PackageDriver command line option, 20
MACAddress (*System attribute*), 43
MarkedCount (*SystemVarsList attribute*), 90
MDNS() (*built-in function*), 120
ModifyAt() (*SystemVarsList method*), 93
MulticastUDP() (*built-in function*), 79

O

-o
PackageDriver command line option, 20
OnAuthenticationFailed() (*SSH method*), 119
OnAuthenticationFailedFunc (*SSH attribute*), 113
OnAuthenticationOK() (*SSH method*), 118
OnAuthenticationOKFunc (*SSH attribute*), 113
OnClientConnect() (*TCPServer method*), 66
OnClientConnectFunc (*TCPServer attribute*), 64
OnClientDisconnect() (*TCPServer method*), 66
OnClientDisconnectFunc (*TCPServer attribute*), 64
OnCommRX() (*HTTP method*), 74
OnCommRX() (*MulticastUDP method*), 80
OnCommRX() (*Serial method*), 59
OnCommRx() (*SSH method*), 119
OnCommRX() (*TCP method*), 63
OnCommRX() (*TCPServer method*), 66
OnCommRX() (*UDP method*), 78
OnComplete() (*MDNS method*), 122
OnConnect() (*HTTP method*), 73

OnConnect() (*SSH method*), 117
OnConnect() (*TCP method*), 62
OnConnectFailed() (*HTTP method*), 73
OnConnectFailed() (*SSH method*), 117
OnConnectFailedFunc (*HTTP attribute*), 68
OnConnectFailedFunc (*SSH attribute*), 112
OnConnectFunc (*HTTP attribute*), 67
OnConnectFunc (*SSH attribute*), 112
OnConnectFunc (*TCP attribute*), 60
OnDisconnect() (*HTTP method*), 73
OnDisconnect() (*SSH method*), 117
OnDisconnect() (*TCP method*), 62
OnDisconnectFunc (*HTTP attribute*), 68
OnDisconnectFunc (*SSH attribute*), 113
OnDisconnectFunc (*TCP attribute*), 60
OnDiscover() (*MDNS method*), 121
OnHandshakeFailed() (*SSH method*), 118
OnHandshakeFailedFunc (*SSH attribute*), 113
OnHandshakeOK() (*SSH method*), 117
OnHandshakeOKFunc (*SSH attribute*), 113
OnOneWayTxFunc (*Serial attribute*), 58
OnPingComplete() (*Ping method*), 111
OnPingCompleteFunc (*Ping attribute*), 109
OnPingFailed() (*Ping method*), 111
OnPingFailedFunc (*Ping attribute*), 108
OnPingOK() (*Ping method*), 110
OnPingOKFunc (*Ping attribute*), 108
OnScheduledEvent() (*ScheduledEvent method*), 87
OnScrollInfo() (*SystemVarsList method*), 96
OnScrollInfoFunc (*SystemVarsList attribute*), 90
OnSendHeartbeat() (*Comm method*), 57
OnShutdownFunc (*System attribute*), 43
OnSSLHandshakeFailed() (*HTTP method*), 75
OnSSLHandshakeFailedFunc (*HTTP attribute*), 68
OnSSLHandshakeOK() (*HTTP method*), 74
OnSSLHandshakeOKFunc (*HTTP attribute*), 68
OnSysVarChangeFunc (*SystemVars attribute*), 88
OnTimer() (*Timer method*), 83
OnWebsocketPing() (*HTTP method*), 76
OnWebsocketPingFunc (*HTTP attribute*), 69
OnWebsocketUpgradeFailed() (*HTTP method*), 76
OnWebsocketUpgradeFailedFunc (*HTTP attribute*), 69
OnWebsocketUpgradeOK() (*HTTP method*), 75
OnWebsocketUpgradeOKFunc (*HTTP attribute*), 69
Open() (*HTTP method*), 70
Open() (*SSH method*), 114
Open() (*SystemVarsList method*), 91
Open() (*TCP method*), 61
OpenState (*HTTP attribute*), 67
OpenState (*SSH attribute*), 112
OpenState (*TCP attribute*), 60

P

PackageDriver command line option
  -i, 20
  -m, 20
  -o, 20
parameter (*functions/category/function element*), 38
PBKDF2() (*Crypto method*), 106
Ping() (*built-in function*), 109
Ping() (*System method*), 51
Port (*TCPServer attribute*), 64
Print() (*System method*), 45
PrintMultiline() (*System method*), 45
processorScript (*driverManifest/driver element*), 27

R

Read() (*Comm method*), 54
Read() (*Persistence method*), 97
Read() (*SystemVars method*), 89
ReadAt() (*SystemVarsList method*), 93
RemoveAll() (*SystemVarsList method*), 94
RemoveAllFilters() (*MulticastUDP method*), 80
RemoveAt() (*SystemVarsList method*), 94
RemoveFilter() (*MulticastUDP method*), 79
RemoveMarked() (*SystemVarsList method*), 95
RemoveSubscription() (*SystemVars method*), 89
Reschedule() (*ScheduledEvent method*), 86
resource (*driverManifest/driver element*), 28
RSADecrypt() (*Crypto method*), 102

RSAEncrypt() (*Crypto method*), 101
RSAGenerateKey() (*Crypto method*), 99
RSASign() (*Crypto method*), 99
RSAVerify() (*Crypto method*), 100
RunSystemMacro() (*System method*), 46

S

Save() (*Persistence method*), 98
ScheduledEvent() (*built-in function*), 8486
Serial() (*built-in function*), 58
SetIndexes() (*SystemVarsList method*), 95
SetMarked() (*SystemVarsList method*), 94
SetPriority() (*System method*), 47
setting (*configuration/category element*), 31
SetTxInterMsgDelay() (*Comm method*), 55
SignalEvent() (*System method*), 47
Size (*SystemVarsList attribute*), 90
Sleep() (*System method*), 45
source (*device/sources element*), 28
sources (*device element*), 28
SSH() (*built-in function*), 114
SSLHandshakeTimeout (*HTTP attribute*), 69
Start() (*Ping method*), 110
Start() (*Timer method*), 82
StartHandshake() (*SSH method*), 115
StartSSLHandshake() (*HTTP method*), 71
StartUPnPScan() (*System method*), 47
State (*Timer attribute*), 81
Stop() (*Ping method*), 110
Stop() (*Timer method*), 82
SystemVarsList() (*built-in function*), 91

T

TCP() (*built-in function*), 61
TCPServer() (*built-in function*), 64
template (*device/sources/source/templates element*), 29
templates (*device/sources/source element*), 29
Timeout (*MDNS attribute*), 120
Timeout (*Ping attribute*), 108
Timer() (*built-in function*), 82
toString() (*Util method*), 98
TxQueueDepth (*Comm attribute*), 53

U

UDP() (*built-in function*), 77
Uncompress() (*System method*), 49
UpgradeWebsocket() (*HTTP method*), 72
UseHandleInCallbacks (*Comm attribute*), 53
UseHandleInCallbacks (*Ping attribute*), 108
UseHandleInCallbacks (*ScheduledEvent attribute*), 84
UseHandleInCallbacks (*Timer attribute*), 81

V

variable (*variables/category element*), 34
variables (*element*), 34
Version (*System attribute*), 43

W

WaitForRx() (*Comm method*), 54
WebsocketAddHeader() (*HTTP method*), 72
Write() (*Comm method*), 54
Write() (*Persistence method*), 97
Write() (*SystemVars method*), 88
Write() (*TCPServer method*), 65
WriteToAddress() (*UDP method*), 77